T.C.
KONYA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

LİSANS BİTİRME PROJESİ

ÇAMAŞIR MAKİNESİ KONTROL BİRİMİNİN VHDL TEMELLİ TASARIMI VE TESTLERİNİN GERÇEKLEŞTİRİLMESİ

Hazırlayan
Yasemin KURT
211202045

Danışman
Doç. Dr. Hasan KOYUNCU

2025
Konya

T.C.
KONYA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

LİSANS BİTİRME PROJESİ

ÇAMAŞIR MAKİNESİ KONTROL BİRİMİNİN VHDL TEMELLİ TASARIMI VE TESTLERİNİN GERÇEKLEŞTİRİLMESİ

Hazırlayan

Yasemin KURT

211202045

Bu çalışma, ……/…../202... tarihinde Komisyonumuz tarafından Elektrik Elektronik Mühendisliği Bölümü LİSANS BİTİRME PROJESİ olarak kabul edilmiştir.

Prof. Dr. Salih GÜNEŞ Prof. Dr. Rahime CEYLAN

Doç. Dr. Hasan KOYUNCU

Prof. Dr. Seral ÖZŞEN
Elk. Elt. Müh. Böl.

ÖZET

ÇAMAŞIR MAKİNESİ KONTROL BİRİMİNİN VHDL TEMELLİ TASARIMI VE TESTLERİNİN GERÇEKLEŞTİRİLMESİ

Bu çalışmada, çamaşır makinesi kontrol biriminin donanım tanımlama dili VHDL (Very High Speed Integrated Circuit Hardware Description Language) kullanılarak tasarlanması ve test edilmesi amaçlanmıştır. Günümüzde çamaşır makineleri çoğunlukla mikrodenetleyici tabanlı gömülü sistemler ile kontrol edilirken, bu çalışmada kontrol algoritması donanım seviyesinde, Sonlu Durum Makinesi (Finite State Machine – FSM) temelli bir yaklaşımla modellenmiştir.

Tasarım sürecinde; kullanıcıdan alınan program seçimi, sıcaklık ayarı, kapak ve deterjan kontrol bilgileri ile birlikte yıkama, durulama ve sıkma gibi temel işlem adımları FSM yapısına dahil edilmiştir. Seçilen programa bağlı olarak işlem süreleri değişmekte olup, bu süreler sayaç tabanlı zamanlama mekanizmaları ile kontrol edilmektedir. Ayrıca, ön yıkama ve ekstra durulama gibi kullanıcıya bağlı ek işlemler sisteme opsiyonel olarak entegre edilmiştir. FSM yapısı, tüm işlem adımlarını belirli bir sırayla ve tanımlı süreler içerisinde gerçekleştirecek şekilde yapılandırılmıştır.

Tasarım ve doğrulama çalışmaları Vivado geliştirme ortamı kullanılarak gerçekleştirilmiş, oluşturulan testbench yapısı aracılığıyla davranışsal simülasyonlar yapılmıştır. Simülasyon sonuçları, tanımlanan giriş koşulları altında sistemin durum geçişlerini ve zamanlama davranışlarını beklenen şekilde gerçekleştirdiğini göstermiştir. Sonuç olarak, bu çalışma çamaşır makinesi kontrol algoritmasının FSM tabanlı ve donanımsal olarak VHDL ile modellenebileceğini ortaya koymakta; FPGA tabanlı kontrol sistemleri için eğitsel ve araştırma amaçlı bir referans uygulama sunmaktadır.

Anahtar Kelimeler:

VHDL, FPGA, Finite State Machine (FSM), Vivado, çamaşır makinesi kontrolü, dijital sistem tasarımı, zamanlayıcı sayaç, donanım tabanlı kontrol.

ÖNSÖZ

Lisans eğitimim süresince edindiğim teorik bilgileri pratiğe dökebileceğim, dijital sistem tasarımı ve donanım tanımlama dilleri alanında kendimi geliştirebileceğim ve mühendislik bakış açımı pekiştirebileceğim bir çalışma yapma hedefiyle bu bitirme projesini gerçekleştirdim. Bu çalışma ile, literatürde çoğunlukla yazılım tabanlı çözümlerle ele alınan kontrol sistemlerinin, FSM tabanlı ve donanımsal bir yaklaşımla da modellenebileceği ortaya konulmuştur.

Bu çalışma sürecinde karşılaşılan teknik problemler ve çözüm aşamaları, analitik düşünme ve problem çözme becerilerimin gelişmesine katkı sağlamıştır.

Bu çalışmanın gerçekleştirilmesi sürecinde bilgi ve deneyimleriyle yol gösteren, desteğini esirgemeyen ve öğrencisi olmaktan gurur duyduğum danışman hocam Doç. Dr. Hasan KOYUNCU ’ya, eğitim hayatım boyunca emeği geçen tüm değerli hocalarıma, her zaman yanımda olan aileme ve desteklerini esirgemeyen arkadaşlarıma teşekkür ederim.

Bu çalışmanın, dijital sistemler ve FPGA tabanlı uygulamalar alanında çalışma yapan araştırmacılar ve uygulayıcılar için faydalı bir kaynak olmasını temenni ederim.

Yasemin Kurt
Konya Teknik Üniversitesi
Elektrik-Elektronik Mühendisliği Bölümü
Konya - 2025

İÇİNDEKİLER

ÖZET i
ÖNSÖZ ii
İÇİNDEKİLER iii
ŞEKİLLER LİSTESİ v
KISALTMALAR vii
1. GİRİŞ 8
1.1. Projenin Amacı ve Kapsam 8
1.2. İlgili Teknoloji Alanları 9
1.3. Kullanılan Yöntem 9
2. KAYNAK ARAŞTIRMASI 10
3. MATERYAL VE METOT (veya TEORİK ESASLAR) 12
3.1. Tasarım Hakkında Genel Bilgiler 12
3.1.1. Program ve sıcaklık modları 12
3.1.2. Programların işlem süreleri 13
3.1.3. İşlemlerin gerçekleşme adımları 14
3.2. Main Kod Mimarisi 16
3.2.1. Kütüphane ve paket tanımlamaları 16
3.2.2. Entity 17
3.2.3. Architecture (Mimari) 18
3.2.4. Procedure (Alt program tanımı) 20
3.2.5. Process 22
3.3. Testbench Mimarisi 33
3.3.1. Kütüphane ve entity tanımlamaları 34
3.3.2. Architecture içerisinde yer alan component tanımlamaları 34

3.3.3. Architecture içerisinde yer alan sinyal ve sabit tanımlamaları 35

3.3.4. Ana tasarımın testbench içerisinde örneklenmesi (Port map)..... 36

3.3.5. Uyarı (Stimulus) süreci ve örnek durumlar üzerinden gözlem 37

3.3.6. Saat (Clock) üretimi ve simülasyonun sonlandırılması 39

4. ARAŞTIRMA (veya DENEY) SONUÇLARI..... 40

4.1. RTL Devre Yapısı 40

4.1.1. Genel RTL şeması..... 41

4.1.2. FSM yapısına ait RTL görünümü 42

4.1.3. Sayaç ve kontrol bloklarına ait RTL görünümü..... 43

4.1.4. mevcut_sure_durumu'nun RTL seviyesinde oluşturulması..... 44

4.1.5. Kullanıcı seçimlerine göre yıkama, durulama ve sıkma sürelerinin RTL seviyesinde belirlenmesi 45

4.2. Testbench Simülasyon Sonuçları ve Dalga Formlarının İncelenmesi 46

4.2.1. Reset durumu simülasyon sonuçları 47

4.2.2. Pamuklu programı (20°C, ön yıkama ve ekstra durulama aktif) simülasyon sonuçları 49

5. TARTIŞMA 57

6. SONUÇ VE ÖNERİLER 59

7. KAYNAKÇA 61

EKLER 63

ŞEKİLLER LİSTESİ

Şekil 3. 1. İşlem Adımları 15
Şekil 3. 2. Kütüphane ve paket tanımlamaları 17
Şekil 3. 3. Tasarıma ait entity yapısı 17
Şekil 3. 4. Tasarıma ait architecture yapısı 19
Şekil 3. 5. Tasarıma ait procedure yapısı 20
Şekil 3. 6. Pamuklu programı için atanan sıcaklık değerlerinde işletilecek olan gecikme süresi atamaları 21
Şekil 3. 7. FSM tabanlı process yapısına ait kod bloğu 23
Şekil 3. 8. Reset durumunda gerçekleştirilen atamalar 24
Şekil 3. 9. Kapak kapalıyken gerçekleştirilen başlangıç atamaları 24
Şekil 3. 10. basla durumuna ait işlem adımları 25
Şekil 3. 11. su_al1 durumunda yürütülecek işlemleri içeren kod bloğu 26
Şekil 3. 12. on_yika durumunda yürütülecek işlemleri içeren kod bloğu 27
Şekil 3. 13. su_ver1 durumunda yürütülecek işlemleri içeren kod bloğu 27
Şekil 3. 14. su_al2 durumunda yürütülecek işlemleri içeren kod bloğu 28
Şekil 3. 15. yika durumunda yürütülecek işlemleri içeren kod bloğu 28
Şekil 3. 16. det_al durumunda yürütülecek işlemleri içeren kod bloğu 28
Şekil 3. 17. su_ver2 durumunda yürütülecek işlemleri içeren kod bloğu 29
Şekil 3. 18. su_al3 durumunda yürütülecek işlemleri içeren kod bloğu 29
Şekil 3. 19. durula durumunda yürütülecek işlemleri içeren kod bloğu 29
Şekil 3. 20. su_ver3 durumunda yürütülecek işlemleri içeren kod bloğu 30
Şekil 3. 21. su_al4 durumunda yürütülecek işlemleri içeren kod bloğu 30
Şekil 3. 22. eks_durula durumunda yürütülecek işlemleri içeren kod bloğu. 31
Şekil 3. 23. su_ver4 durumunda yürütülecek işlemleri içeren kod bloğu 31
Şekil 3. 24. skma durumunda yürütülecek işlemleri içeren kod bloğu 32
Şekil 3. 25. others durumunda yürütülecek işlemleri içeren kod bloğu 32
Şekil 3. 26. kapak açıksa yürütülecek işlemleri içeren kod bloğu 33
Şekil 3. 27. Testbench için kütüphane, paket ve entity tanımlamalarını içeren kod bloğu 34
Şekil 3. 28. Testbench'e ait component yapısı 35

Şekil 3. 29. Testbench architecture sinyal ve sabit tanımlarını içeren kod bloğu 36
Şekil 3. 30. Testbench port map yapısını içeren kod bloğu 37
Şekil 3. 31. Testbench stimulus - reset durumu kod bloğu 38
Şekil 3. 32. Testbench stimulus - pamuklu 20 °C programı için ön yıkama ve ekstra durulama seçeneklerinin aktif olduğu durum kod bloğu 38
Şekil 3. 33. Testbench içerisinde clock sinyalinin üretildiği ve simülasyonun kontrol edildiği kod bloğu 39

Şekil 4. 1. RTL analizi ile elde edilen devre yapısı 41
Şekil 4. 2. FSM yapısına ait RTL devre örneği 42
Şekil 4. 3. Sayaç ve kontrol bloklarına ait RTL devre örneği 44
Şekil 4. 4. Mevcut_sure_durumu'na ait RTL devre örneği 45
Şekil 4. 5. Yıkama, durulama, sıkma durumlarına ait RTL devre örneği 46
Şekil 4. 6. Reset durumu simülasyon çıktısı – 1 47
Şekil 4. 7. Reset durumu simülasyon çıktısı - 2 48
Şekil 4. 8. Ön yıkama öncesi su alma aşamasına ait dalga formu 50
Şekil 4. 9. Ön yıkama aşamasına ait dalga formu 50
Şekil 4. 10. Ön yıkama sonrası su tahliye aşamasına ait dalga formu 51
Şekil 4. 11. Ana yıkama öncesi temiz su alma aşamasına ait dalga formu 51
Şekil 4. 12. Ana yıkama aşamasına ait dalga formu 52
Şekil 4. 13. Deterjan alma aşamasına ait dalga formu 52
Şekil 4. 14. Yıkama sonrası su tahliye aşamasına ait dalga formu 53
Şekil 4. 15. Durulama öncesi su alma aşamasına ait dalga formu 53
Şekil 4. 16. Durulama aşamasına ait dalga formu 54
Şekil 4. 17. Durulama sonrası su tahliye aşamasına ait dalga formu 54
Şekil 4. 18. Ekstra durulama için su alma aşamasına ait dalga formu 55
Şekil 4. 19. Ekstra durulama aşamasına ait dalga formu 55
Şekil 4. 20. Ekstra durulama sonrası su tahliye aşamasına ait dalga formu 56
Şekil 4. 21. Sıkma aşaması ve program bitişine ait dalga formu 56

KISALTMALAR

- FSM : Finite State Machine (Sonlu Durum Makinesi)
- RTL : Register Transfer Level (Yazmaç Aktarım Seviyesi)
- FPGA : Field Programmable Gate Array (Alan Programlanabilir Kapı Dizisi)
- VHDL : Very High Speed Integrated Circuit Hardware Description Language
- MUX : Multiplexer (Çoklayıcı)
- ns : Nanosaniye
- rst : Reset (Sıfırlama Sinyali)
- CLK : Clock (Saat Sinyali)
- IEEE : Institute of Electrical and Electronics Engineers

1. GİRİŞ

Günümüzde teknolojinin hızlı gelişimiyle birlikte ev aletleri daha akıllı, verimli, kullanıcı dostu ve özelleştirilebilir hale gelmiştir. Bu cihazlardan biri olan çamaşır makineleri, farklı program seçenekleri ve gelişmiş kontrol mekanizmaları sayesinde kullanıcılara önemli kolaylıklar sağlamaktadır. Çamaşır makinelerinin güvenli ve verimli bir şekilde çalışabilmesi, bünyelerinde yer alan kontrol birimlerinin doğruluğu ve güvenilirliği ile doğrudan ilişkilidir.

Geleneksel çamaşır makinesi kontrol sistemleri çoğunlukla mikrodenetleyiciler üzerinde çalışan yazılım tabanlı yapılar ile tasarlanmaktadır. Bu tez çalışmasında ise donanım tanımlama dili olan VHDL kullanılarak, FSM (Finite State Machine) temelli ve tamamen donanımsal bir çamaşır makinesi kontrol birimi tasarlanmıştır. Önerilen yaklaşımda kontrol algoritması, durum geçişleri ve zamanlama işlemleri donanım seviyesinde kurgulanmıştır. Bu sayede paralel işlem yapabilme yeteneği elde edilmiş, sistemin tepkime süresi kısaltılmış ve deterministik bir çalışma davranışı sağlanmıştır.

1.1. Projenin Amacı ve Kapsam

Bu projenin temel amacı, bir çamaşır makinesinin kontrol algoritmasını dijital devre düzeyinde VHDL ile modelleyerek donanım temelli bir kontrol birimi geliştirmektir. Bu kapsamda; kullanıcı tarafından belirlenen program seçimi ve sıcaklık ayarlarının yanı sıra deterjan ve kapak kontrolü gibi temel güvenlik ve kontrol işlevleri tasarıma dahil edilmiştir.

Çamaşır makinesinin yıkama, durulama ve sıkma aşamaları FSM mantığı kullanılarak sıralı ve kontrollü bir biçimde gerçekleştirilmiştir. Her bir işlem aşaması, zamanlayıcı sayaçlar aracılığıyla belirlenen süreler boyunca yürütülmektedir. Ayrıca ön yıkama ve ekstra durulama gibi opsiyonel işlem adımları, kullanıcı tercihi doğrultusunda işletilebilecek şekilde tasarlanmıştır. Geliştirilen sistem, Vivado tasarım ortamında oluşturulmuş ve testbench yapıları aracılığıyla simülasyon ortamında doğrulanmıştır.

1.2. İlgili Teknoloji Alanları

Bu proje aşağıdaki teknoloji alanları ile doğrudan ilişkilidir:

- **Dijital Sistem Tasarımı:** VHDL kullanılarak donanım düzeyinde sistem modelleme, mantık devreleri ve FSM tabanlı kontrol yapıları gerçekleştirilmiştir.
- **Gömülü Sistemler:** Donanım tabanlı kontrol birimleri aracılığıyla gerçek dünya uygulamalarına yönelik sistem tasarımı ele alınmıştır.
- **FPGA Tabanlı Tasarım:** Xilinx Vivado ortamı kullanılarak FPGA üzerinde tasarım, simülasyon ve doğrulama süreçleri yürütülmüştür.
- **Ev Otomasyonu ve Akıllı Cihazlar:** Akıllı ev aletlerinde kullanılan kontrol algoritmaları, özelleştirilebilir donanım tabanlı bir yapı üzerinden modellenmiştir.

1.3. Kullanılan Yöntem

Bu çalışmada FSM yaklaşımı temel alınarak, çamaşır makinesi kontrol biriminin durum geçişleri belirli kurallar ve süre koşullarına bağlı olarak gerçekleştirilmiştir. Yıkama, durulama, sıkma gibi temel işlem aşamaları için zamanlayıcı sayaçlar kullanılmış; durum geçişleri bu sayaçların değerlerine göre kontrol edilmiştir.

Kullanıcının seçtiği programa bağlı olarak işlem aşamalarının süreleri değişmekte, işlem adımlarının genel sırası ise tüm programlar için sabit tutulmaktadır. Ön yıkama ve ekstra durulama gibi opsiyonel işlem adımları FSM yapısı içerisinde tanımlı durumlar olarak yer almakta olup, ilgili özelliklerin kullanıcı tarafından aktif edilmediği durumlarda bu adımlar işlem gerçekleştirilmeden doğrudan bir sonraki duruma geçilerek atlanmaktadır.

Tasarım VHDL ile kodlanmış, Vivado ortamında gerçekleştirilen davranışsal simülasyonlar aracılığıyla tüm durum geçişlerinin ve zamanlama davranışlarının doğruluğu teyit edilmiştir.

2. KAYNAK ARAŞTIRMASI

Çamaşır makineleri gibi ev aletlerinin kontrol sistemleri ile ilgili literatür incelendiğinde, bu tür sistemlerin büyük çoğunlukla mikrodenetleyici tabanlı gömülü yazılımlar kullanılarak gerçekleştirildiği görülmektedir. Mevcut çalışmalarda kontrol algoritmaları genellikle yazılım seviyesinde tanımlanmakta; program akışı, zamanlama ve durum geçişleri işlemci üzerinde çalışan yazılımlar aracılığıyla yönetilmektedir. Bu yaklaşımlarda sistem davranışı çoğunlukla yazılım soyutlamaları üzerinden ele alınmakta, donanım seviyesindeki durum geçişleri ve zamanlama mekanizmaları ayrıntılı olarak modellenmemektedir. Bu durum, kontrol algoritmasının donanımsal karşılığının analiz edilmesini ve sistem davranışının zamansal açıdan değerlendirilmesini zorlaştırmaktadır.

Dijital sistem tasarımı alanında yapılan çalışmalarda ise, kontrol problemlerinin FSM tabanlı yaklaşımlar ile modellenmesinin, sistem davranışının açık ve deterministik bir şekilde tanımlanmasına olanak sağladığı vurgulanmaktadır. FSM tabanlı tasarım yöntemleri; durum geçişlerinin açıkça tanımlanabilmesi, zamanlama koşullarının net bir şekilde kontrol edilebilmesi ve sistemin doğrulanabilirliğinin artırılması gibi avantajlar sunmaktadır. Bu bağlamda, dijital kontrol sistemlerinin donanım tanımlama dilleri kullanılarak modellenmesi, özellikle FPGA tabanlı uygulamalarda tercih edilen bir yöntem haline gelmiştir.

Literatürde yer alan dijital sistem ve FSM tabanlı tasarım çalışmalarında, senkron durum makineleri, sayaç tabanlı zamanlama yapıları ve donanım tanımlama dilleri ile kontrol algoritmalarının modellenmesi detaylı olarak ele alınmaktadır. Bu çalışmalarda FSM yapısının, kontrol edilecek sistemin tüm işlem adımlarını belirli bir sıra ve süre düzeni içerisinde gerçekleştirecek şekilde yapılandırılabileceği gösterilmiştir. Ancak ev tipi cihazlar özelinde, özellikle çamaşır makineleri için FSM tabanlı ve tamamen donanımsal kontrol birimi tasarımına yönelik çalışmaların sınırlı olduğu görülmektedir. Bu durum, donanım temelli kontrol yaklaşımlarının bu alanda yeterince incelenmediğini ortaya koymaktadır.

Bu tez çalışması, literatürde yaygın olarak kullanılan mikrodenetleyici tabanlı yazılım yaklaşımlarından farklı olarak, çamaşır makinesi kontrol algoritmasının FSM temelli ve donanım seviyesinde modellenmesine odaklanmaktadır. Böylece kontrol

sisteminin durum geçişleri, süre yönetimi ve işlem sıralaması doğrudan donanım üzerinden tanımlanmakta; yazılım tabanlı kontrol yapılarında karşılaşılabilecek zamanlama belirsizliklerinin önüne geçilmektedir.

Bu yönüyle çalışma, ev tipi cihazların kontrolüne yönelik donanım tabanlı FSM uygulamalarına örnek teşkil ederek literatüre tamamlayıcı bir katkı sunmaktadır.

3. MATERYAL VE METOT (veya TEORİK ESASLAR)

3.1. Tasarım Hakkında Genel Bilgiler

Tasarımın genel özellikleri, alt başlıklar halinde detaylandırılmıştır.

3.1.1. Program ve sıcaklık modları

Bu çalışmada geliştirilen çamaşır makinesi kontrol birimi, kullanıcıdan alınan program ve sıcaklık seçimlerine bağlı olarak farklı yıkama senaryolarını destekleyecek şekilde tasarlanmıştır. Kullanıcı, sistemin başlangıcında program türünü ve sıcaklık seviyesini belirlemekte; kontrol birimi bu seçimlere uygun olarak FSM tabanlı işlem akışını yürütmektedir.

Tasarlanan programlar ve sıcaklık değerleri, günlük kullanımda yaygın olarak tercih edilen çamaşır yıkama modları esas alınarak modellenmiştir [1–6].

Aşağıda tasarımda kullanılan program türleri ve sıcaklık modları maddeler halinde verilmiştir:

- Pamuklu => 20°C, 40°C, 60°C ve 90°C
- Sentetik => 30°C ve 40°C
- Karışık => 40°C
- Hassas/İpek => 30°C
- Elde Yıkama => 30°C
- Kot/Koyu Renk => 40°C
- Alerji/Bebek => 60°C
- Spor => 30°C
- Gömlek => 40°C
- Hızlı Yıkama => 40°C ve soğuk (❄)

Belirlenen bu program–sıcaklık eşleşmeleri, kontrol birimi tarafından geçersiz seçimlerin engellenmesini sağlayacak şekilde FSM yapısına entegre edilmiştir.

3.1.2. Programların işlem süreleri

Tüm program ve sıcaklık seçimleri için ortak olarak gerçekleştirilen bazı işlem adımları sabit süreli olarak tanımlanmıştır. Ön yıkama ve ekstra durulama gibi kullanıcı tercihine bağlı ek işlemler de benzer şekilde sabit süreli aşamalar olarak modellenmiştir.

Bu çalışmada işlem süreleri gerçek zaman birimleri cinsinden doğrudan ifade edilmemiş, bunun yerine her bir işlem adımı için FPGA sistem saatine bağlı sayaç değerleri kullanılmıştır. Kavramsal olarak ifade edilen süreler, FSM yapısı içerisinde tanımlanan sayaçların belirli sayıda saat darbesi (clock pulse) sayması esasına göre gerçekleştirilmiştir. Böylece zamanlama işlemleri, yazılım tabanlı gecikmelere ihtiyaç duyulmadan, tamamen donanım seviyesinde kontrol edilmiştir.

Sabit süreye sahip işlem adımları ve bu adımların kavramsal süre karşılıkları Tablo-1'de sunulmuştur. Bu süreler, FPGA üzerinde uygulamada sistemin saat frekansına karşılık gelen sayaç değerleri ile temsil edilmektedir.

Gerçekleşme Durumu	**İşlem**	**Kavramsal Süre Karşılığı**
Ön Yıkama Seçilmiş İse	Ön Yıkama Öncesi Su Alma	5 tetikleme
Ön Yıkama Seçilmiş İse	Ön Yıkama	15 tetikleme
Ön Yıkama Seçilmiş İse	Ön Yıkama Sonrası Su Verme	5 tetikleme
Her Zaman	Yıkama Öncesi Su Alma	15 tetikleme
Her Zaman	Deterjan Alma	3 tetikleme
Her Zaman	Yıkama Sonrası Su Verme	15 tetikleme
Her Zaman	Durulama Öncesi Su Alma	15 tetikleme
Her Zaman	Durulama Sonrası Su Verme	15 tetikleme
Ekstra Durulama Seçilmiş İse	Ekstra Durulama Öncesi Su Alma	5 tetikleme
Ekstra Durulama Seçilmiş İse	Ekstra Durulama	15 tetikleme
Ekstra Durulama Seçilmiş İse	Ekstra Durulama Sonrası Su Verme	5 tetikleme

Tablo 1: Sabit işlemler ve kavramsal süre karşılıkları

Program ve sıcaklık tercihlerine bağlı olarak değişkenlik gösteren yıkama, durulama ve sıkma işlemlerine ait kavramsal süre karşılıkları ise Tablo-2'de verilmiştir. Bu değerler, FSM içerisinde ilgili durumlara atanmış sayaç hedefleri ile uygulanmaktadır.

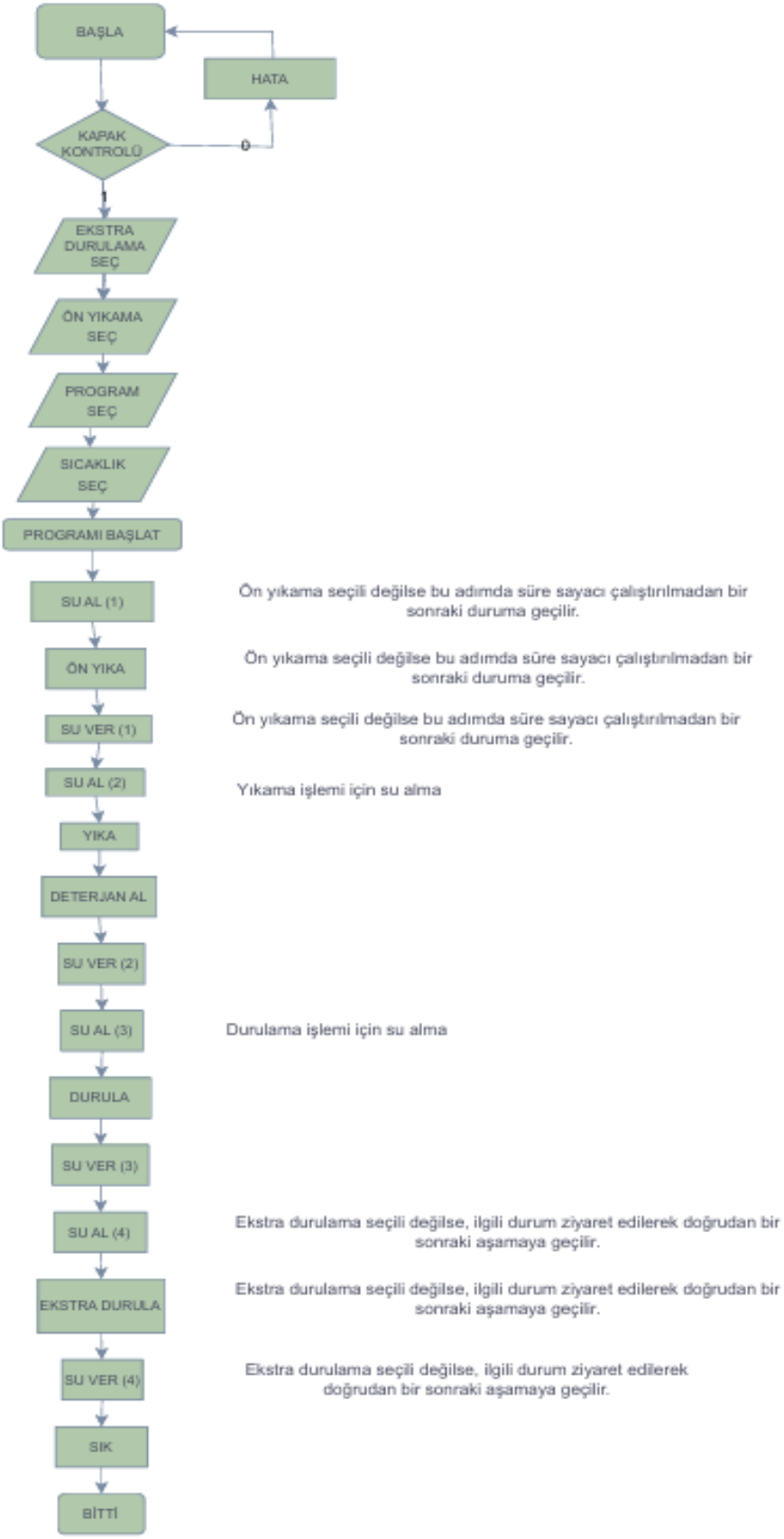
Program	**Sıcaklık**	**Yıkama**	**Durulama**	**Sıkma**
Pamuklu	20°	40 tetikleme	10 tetikleme	10 tetikleme
Pamuklu	40°	45 tetikleme	10 tetikleme	10 tetikleme
Pamuklu	60°	55 tetikleme	10 tetikleme	10 tetikleme
Pamuklu	90°	70 tetikleme	10 tetikleme	10 tetikleme
Sentetik	30°	35 tetikleme	10 tetikleme	8 tetikleme
Sentetik	40°	35 tetikleme	10 tetikleme	8 tetikleme
Karışık	40°	40 tetikleme	10 tetikleme	10 tetikleme
Hassas/İpek	30°	25 tetikleme	8 tetikleme	5 tetikleme
Elde Yıkama	30°	35 tetikleme	8 tetikleme	5 tetikleme
Kot/Koyu Renk	40°	50 tetikleme	10 tetikleme	12 tetikleme
Alerji/Bebek	60°	55 tetikleme	16 tetikleme	12 tetikleme
Spor	30°	30 tetikleme	10 tetikleme	8 tetikleme
Gömlek	40°	35 tetikleme	8 tetikleme	8 tetikleme
Hızlı Yıkama	40°	20 tetikleme	5 tetikleme	5 tetikleme
Hızlı Yıkama	❄°	14 tetikleme	5 tetikleme	5 tetikleme

Tablo 2: Yıkama, durulama, sıkma işlemlerinin kavramsal süre karşılığı

Belirlenen kavramsal süre karşılıkları, farklı üreticilere ait çamaşır makinelerinin teknik dokümanları incelenerek ve karşılaştırmalı değerlendirmeler yapılarak oluşturulmuştur [1-6]. FPGA üzerinde gerçekleştirilen uygulamada bu süreler, kullanılan sistem saat frekansına bağlı olarak ölçeklenebilir niteliktedir.

3.1.3. İşlemlerin gerçekleşme adımları

Program çalışmaya başladıktan sonra gerçekleşecek işlem adımları şu şekildedir:



Şekil 3. 1. İşlem Adımları

3.2. Main Kod Mimarisi

Geliştirilen VHDL tabanlı çamaşır makinesi kontrol biriminin ana kod yapısı, FSM mimarisi esas alınarak oluşturulmuştur. Bu mimari sayesinde sistemin çalışma süreci belirli durumlar altında tanımlanmış, işlem adımları ardışık ve kontrollü bir şekilde yürütülmüştür. FSM yapısı, çamaşır makinesinin çalışma mantığını adım adım modellemeye olanak sağlamaktadır.

Tasarlanan kontrol biriminde durumlar arası geçişler; kullanıcıdan alınan program ve ek özellik seçimleri ile birlikte zamanlama koşullarına bağlı olarak gerçekleştirilmektedir. Zamanlama işlemleri, sistem saat sinyaline bağlı çalışan sayaçlar aracılığıyla kontrol edilmekte olup, her bir işlem adımı belirlenen saat darbesi sayısı tamamlandığında bir sonraki duruma geçmektedir. Bu yaklaşım sayesinde kontrol algoritmasının tamamı donanım seviyesinde tanımlanmış ve deterministik bir çalışma davranışı elde edilmiştir.

3.2.1. Kütüphane ve paket tanımlamaları

VHDL tabanlı dijital sistem tasarımlarında, mantıksal sinyallerin tanımlanması, kontrol yapılarının oluşturulması ve sistem davranışının doğru şekilde modellenebilmesi için standart kütüphane ve paket tanımlamalarının yapılması gerekmektedir. Bu çalışmada geliştirilen çamaşır makinesi kontrol biriminin tasarım sürecinde, IEEE standartlarına ait kütüphaneler kullanılarak donanım tanımlama dili kurallarına uygun bir yapı oluşturulmuştur.

Kullanılan kütüphane ve paketler sayesinde, sistemin giriş–çıkış sinyalleri, FSM durumları ve kontrol işaretleri açık, tutarlı ve okunabilir bir biçimde modellenmiştir. Ayrıca bu tanımlamalar, kodun sentezlenebilirliğini arttırmış ve tasarımın FPGA üzerinde hatasız bir şekilde çalışmasına katkı sağlamıştır.

Kod içerisinde yer alan kütüphane ve paket tanımlamalarını içeren ilgili kod kesiti Şekil 3.2’de gösterilmektedir.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.numeric_std.ALL;
```

Şekil 3. 2. Kütüphane ve paket tanımlamaları

3.2.2. Entity

VHDL tabanlı tasarımlarda entity yapısı, sistemin dış dünya ile olan etkileşimini tanımlayan, giriş–çıkış portlarının tanımlandığı temel bölümdür. Bu çalışmada geliştirilen çamaşır makinesi kontrol biriminde, sistemin kullanıcıdan aldığı seçimler, çalışma koşulları ve kontrol çıktıları tek bir entity yapısı altında tanımlanmıştır.

Tasarlanan tanim isimli entity; kullanıcı tarafından belirlenen program ve sıcaklık seçimlerini, sistemin çalışma durumunu ve zamanlama parametrelerini kapsayacak şekilde yapılandırılmıştır. İlgili entity tanımı Şekil 3.3'te gösterilmektedir.

```
5  entity tanim is
6      Port (clk,rst : in std_logic;
7      kapak_kontrol : in std_logic;
8      deterjan_kontrol: in std_logic;
9      program_sec_giris : in std_logic_vector(0 to 3);
10     --pamuklu 0000,sentetik 0001, mix 0010, hassas_ip
11     on_yikama_sec : in std_logic;
12     durulama_sec : in std_logic;
13     sicaklik_sec : in std_logic_vector(0 to 2);
14     g_skma : inout integer :=0;
15     g_durula : inout integer :=0 ;
16     g_yika : inout integer :=0;
17     mevcut_sure_durumu : out integer := 0;
18     cikis : out std_logic);
19 end tanim;
```

Şekil 3. 3. Tasarıma ait entity yapısı

Entity içerisinde yer alan portlar, veri yönlerine göre giriş (in), çıkış (out) ve çift yönlü (inout) olarak tanımlanmıştır. Bu ayrım sayesinde sinyallerin sistem içerisindeki kullanım amaçları net bir şekilde belirlenmiştir.

Giriş sinyalleri; sistem saat sinyali ve sıfırlama işareti ile birlikte, kullanıcıdan alınan program seçimi, sıcaklık tercihi, ön yıkama ve ekstra durulama seçenekleri ile kapak ve deterjan kontrol bilgilerini içermektedir. Bu sinyaller, FSM yapısının durum geçişlerini ve işlem sırasını belirleyen temel kontrol girdileri olarak kullanılmaktadır.

Çıkış sinyalleri, sistemin çalışma sürecine ilişkin bilgilerin dış ortama aktarılmasını sağlamaktadır. Bu kapsamda, mevcut süre durumunu temsil eden sinyal ile programın tamamlandığını ifade eden çıkış işareti tanımlanmıştır.

Çift yönlü olarak tanımlanan yıkama, durulama ve sıkma sürelerine ait sinyaller ise, başlangıçta seçilen program parametrelerinin okunması ve FSM sürecinde sayaçların ulaşması gereken hedef değerlerin belirlenmesi amacıyla kullanılmıştır. Bu yapı sayesinde süre parametreleri, kontrol birimi içerisinde dinamik olarak yönetilebilmiştir.

Entity içerisinde kullanılan veri türleri, taşınan bilginin niteliğine göre seçilmiştir. İkili kontrol sinyalleri için mantıksal veri tipleri tercih edilirken, çoklu seçim bilgileri vektörel yapılarla, zamanlama ve sayaç değerleri ise sayısal veri tipleri ile modellenmiştir. Bu yaklaşım, tasarımın okunabilirliğini artırmış ve FSM tabanlı kontrol yapısının sade bir biçimde uygulanmasına katkı sağlamıştır.

3.2.3. Architecture (Mimari)

Geliştirilen çamaşır makinesi kontrol biriminin davranışsal modeli, VHDL'de architecture bloğu içerisinde tanımlanmıştır. Bu bölümde, sistemin çalışma mantığını oluşturan FSM yapısı, durum geçişleri ve zamanlama mekanizması bütüncül bir yapı altında modellenmiştir. Architecture bloğu, kontrol biriminin iç işleyişini belirleyen sinyallerin, durumların ve zamanlayıcı yapıların tanımlandığı ana bölümü oluşturmaktadır.

Tasarımın genel mimarisini oluşturan yapı taşları Şekil 3.4'te gösterilmektedir.

```
architecture davranis of tanim is
    Type islem is (basla, su_al1, su_al2, su_al3, su_al4, su_ver1, su_ver2, su_ver3, su_ver4, on_yika, yika, det_al, durula, eks_durula, skma);
    Signal simdiki_durum:islem;
    Signal sayici5 : integer := 0;
    Signal sayici15 : integer := 0;
    Signal sayici3 : integer := 0;
    Signal sayici_durula : integer := 0;
    Signal sayici_yika : integer := 0 ;
    Signal sayici_skma : integer := 0;
    Signal sonuc_on_yika : std_logic ;
    Signal sonuc_eks_durulama : std_logic ;
    Signal son_sicaklik : std_logic_vector(0 to 2); -- 000 20derece, 001 30derece, 010 40 derece, 011 60 derece, 100 90 derece, 101 * derece
    Signal program_sec_cikis : std_logic_vector(0 to 3);
    Signal deterjan_var_mi : std_logic;
    constant max_sayici_deger5 : integer := 5;
    constant max_sayici_deger15 : integer := 15;
    constant max_sayici_deger3 : integer := 3;
    signal max_sayici_deger_durula : integer := 0;
    signal max_sayici_deger_yika : integer := 0;
    signal max_sayici_deger_skma : integer := 0;
```

Şekil 3. 4. Tasarıma ait architecture yapısı

FSM yapısında yer alan tüm işlem adımları, islem adlı özel bir veri türü (type) içerisinde tanımlanmıştır. Bu yapı sayesinde çamaşır makinesinin çalışması sırasında gerçekleşen tüm aşamalar, belirli bir sıra ve kontrol akışı içerisinde yürütülmektedir. FSM'nin mevcut durumunu temsil edecek olan simdiki_durum sinyali, bu type üzerinden tanımlanmış olup durumlar arası geçişlerin yönetilmesini sağlamaktadır.

Sistemdeki işlem sürelerinin kontrolü, FPGA'nın çalışma frekansı referans alınarak gerçekleştirilen sayaç yapıları üzerinden sağlanmıştır. Bu amaçla, farklı işlem adımları için tanımlanan sayaçlar integer türünde oluşturulmuş ve başlangıçta sıfırlanmıştır. FSM'nin ilgili durumlarında bu sayaçlar saat darbeleri ile artırılarak, her bir işlemin kendisine ait gecikme süresi donanım seviyesinde kontrol edilmiştir. Böylece zamanlama mekanizması yazılım temelli değil, doğrudan donanım tetiklemeli olarak modellenmiştir.

Ön yıkama ve ekstra durulama gibi ek program seçenekleri ile deterjan ve kapak kontrol bilgileri, yalnızca iki durumlu (aktif/pasif) bilgi taşıdıkları için tek bitlik kontrol sinyalleri olarak tanımlanmıştır. Bu sinyaller, FSM'nin durum geçiş koşullarında doğrudan karar mekanizması olarak kullanılmıştır.

Seçilen program ve sıcaklık bilgileri ise birden fazla durumu temsil ettiğinden, çok bitli sinyaller aracılığıyla modellenmiştir. Bunun yanında, her durumda gecikme süresi sabit olan işlem adımlarına ait hedef sayaç değerleri constant tanımları ile sabitlenmiş; bu sayede mimarinin hem okunabilirliği artırılmış hem de zamanlama parametrelerinin merkezi bir yapı üzerinden yönetilmesi sağlanmıştır.

Bu mimari yapı ile FSM tabanlı kontrol birimi; deterministik, kararlı ve senkron bir şekilde çalışacak biçimde tasarlanmıştır.

3.2.4. Procedure (Alt program tanımı)

Architecture bloğu içerisinde gerekli sinyal ve yapı tanımlamaları gerçekleştirildikten sonra, tasarımda kod tekrarını azaltmak ve kontrol biriminin daha modüler bir yapıda oluşturulmasını sağlamak amacıyla bir procedure (alt program) tanımı yapılmıştır. Bu yapı, kullanıcıdan alınan program ve sıcaklık bilgilerinin merkezi bir mekanizma üzerinden değerlendirilmesini ve ilgili zamanlama parametrelerinin sisteme atanmasını sağlamaktadır.

Tasarımda kullanılan programlar isimli procedure tanımı Şekil 3.5'te verilmiştir.

```
procedure programlar(
    signal son_sicaklik : out std_logic_vector;
    signal program_sec_cikis : out std_logic_vector;
    signal program_sec_giris : in std_logic_vector;
    signal sicaklik_sec : in std_logic_vector;
    signal g_durula : inout integer;
    signal g_yika : inout integer;
    signal g_skma : inout integer) is

begin
```

Şekil 3. 5. Tasarıma ait procedure yapısı

Bu procedure yapısında, program_sec_giris ve sicaklik_sec sinyalleri kullanıcı tarafından belirlenen giriş bilgilerini temsil eden giriş parametreleri olarak tanımlanmıştır. Procedure içerisinde hesaplanan veya belirlenen diğer sinyaller ise, sistemin farklı noktalarında kullanılacak değerleri taşıdığından, çıkış yönlü parametreler olarak yapılandırılmıştır.

Procedure yapısının temel amacı; seçilen program türü ve sıcaklık değerine bağlı olarak yıkama, durulama ve sıkma işlemlerine ait maksimum gecikme değerlerinin belirlenmesi ve bu değerlerin FSM tarafından kullanılacak sinyallere atanmasıdır. Böylece program ve sıcaklık kombinasyonlarına bağlı zamanlama bilgileri, tek bir merkezi yapı üzerinden yönetilmiştir.

Bu yaklaşım sayesinde, FSM içerisinde aynı koşullara bağlı süre atamalarının

tekrar edilmesine gerek kalmadan, yalnızca ilgili procedure çağrısı ile gerekli tüm parametrelerin sisteme aktarılması mümkün hâle gelmiştir. Bu durum, kodun okunabilirliğini artırmış ve tasarımın ileride yapılabilecek güncellemelere daha açık bir yapı kazanmasını sağlamıştır.

Şekil 3.6'da örnek olarak Pamuklu programının seçilmesi durumunda çalışan procedure kod bloğu gösterilmektedir.

```
if(program_sec_giris= "0000") then -- seçilen program pamukluysa
            program_sec_cikis <="0000";
            g_durula <= 10; --10dk durulama işlemi
            g_skma <= 10;    --skma için 10dk gecikme

                    if(sicaklik_sec = "000") then --sıcaklık 20 derece ise
                    son_sicaklik <= "000";
                    g_yika <= 40;    --yika için 40dk gecikme

                    elsif(sicaklik_sec = "010") then --sıcaklık 40 derece ise
                    son_sicaklik <= "010";
                    g_yika <= 45;    --yika için 45dk gecikme

                    elsif(sicaklik_sec = "011") then --sıcaklık 60 derece ise
                    son_sicaklik <= "011";
                    g_yika <= 55;    --yika için 55dk gecikme

                    elsif(sicaklik_sec = "100") then --sıcaklık 90 derece ise
                    son_sicaklik <= "100";
                    g_yika <= 70;    --yika için 70dk gecikme
                    end if;
```

Şekil 3. 6. Pamuklu programı için atanan sıcaklık değerlerinde işletilecek olan gecikme süresi atamaları

Bu örnekte hem program türü hem de sıcaklık bilgisi birlikte değerlendirilerek, ilgili program bilgisi program_sec_cikis sinyaline; yıkama, durulama ve sıkma işlemlerine ait gecikme değerleri ise ilgili gecikme sinyallerine atanmıştır. Aynı zamanda seçilen sıcaklık değeri, sistem genelinde kullanılmak üzere son_sicaklik sinyaline aktarılmıştır.

Oluşturulan bu yapı, tüm program ve sıcaklık kombinasyonları için genişletilebilir nitelikte olup, diğer programlar da aynı mantık çerçevesinde benzer şekilde modellenmiştir.

3.2.5. Process

Process yapısı, FSM tabanlı kontrol algoritmasının zamanlama ve durum geçişlerini yöneten temel yapıdır. Tasarımda tüm işlem adımları, saat sinyalinin yükselen kenarına duyarlı ve senkron sıfırlamalı bir process içerisinde gerçekleştirilmiştir. Bu yapı sayesinde sistemin tüm davranışı deterministik, tekrarlanabilir ve sentezlenebilir şekilde modellenmiştir.

Tasarımda kullanılan FSM yapısı; mevcut durumu takip etmek, giriş sinyallerini değerlendirmek, gerekli sayaç işlemlerini yürütmek ve uygun koşullarda bir sonraki duruma geçiş yapmak üzere yapılandırılmıştır. FSM, çamaşır makinesinin gerçek çalışma senaryosuna uygun olacak biçimde işlem adımlarını sıralı olarak yürütmektedir.

Bu çalışmada kullanılan FPGA, Zynq-7000 ailesine ait XC7Z014SCLG484-2 modelidir. Zamanlama işlemleri, kullanılan FPGA'nın çalışma frekansı temel alınarak sayaç tabanlı bir yaklaşımla gerçekleştirilmiştir. Tasarımda 100 MHz frekansında yapılandırılmış saat (clock) sinyali kullanılmıştır. Buna karşılık gelen saat periyodu aşağıdaki şekilde hesaplanmıştır:

$$Tclock = \frac{1}{fclock} = \frac{1}{100 \times 10^6} = 10 \text{ ns} \tag{3.1}$$

Her bir sayaç artışı, clock sinyalinin bir yükselen kenarına karşılık gelmekte olup 10 ns'lik bir zaman ilerlemesini temsil etmektedir. Bu doğrultuda toplam gecikme süresi aşağıdaki ifade ile hesaplanmaktadır:

$$t = N \times T_{clock} \tag{3.2}$$

Bu yaklaşım sayesinde tüm gecikmeler, dakika veya saniye gibi soyut zaman ifadeleri yerine, doğrudan FPGA çalışma frekansına bağlı olarak gerçekleştirilmiştir. Böylece sistem, donanım seviyesinde zaman kontrollü ve güvenilir bir yapı kazanmıştır.

Tasarımda, hesaplanan gecikme değerlerine uygun olacak şekilde sayaçlar kullanılmıştır. İlgili sayaçlar, her bir clock yükselen kenarında bir artırılmakta; sayaç değeri belirlenen maksimum değere ulaştığında sayaç sıfırlanmakta ve FSM yapısı

aracılığıyla bir sonraki duruma geçilmektedir. Sayaç değeri maksimum değere ulaşmadığı sürece ise FSM mevcut durumda kalmaktadır. Bu mekanizma ile gerçek zamanlı gecikme ihtiyacı donanım tabanlı olarak çözülmüştür.

Tasarımda kullanılan process yapısının bir bölümü Şekil 3.7'de gösterilmiştir.

```
process (clk,rst, program_sec_giris, sicaklik_sec, g_durula, g_skma, g_yika)
begin --process için

programlar(
              son_sicaklik,
              program_sec_cikis,
              program_sec_giris,
              sicaklik_sec,
              g_durula, -- inout sinyali
              g_yika, -- inout sinyali
              g_skma -- inout sinyali
          );


max_sayici_deger_skma <= g_skma;
max_sayici_deger_durula <= g_durula;
max_sayici_deger_yika <= g_yika;
```

Şekil 3. 7. FSM tabanlı process yapısına ait kod bloğu

Process, saat darbesi (clk) ve senkron sıfırlama sinyali (rst) temelinde çalışmaktadır. Şekil 3.7'de görüldüğü üzere saat darbesi ve senkron sıfırlama sinyaline ek olarak, kullanıcının seçmiş olduğu program ve sıcaklık bilgileri (program_sec_giris ve sicaklik_sec) ile bu tercihlere bağlı olarak belirlenen gecikme süreleri (g_durula, g_skma ve g_yika) sinyalleri süreci tetikleyen sinyaller olarak tanımlanmıştır.

Process bloğu içerisinde, daha önce tanımlanan programlar isimli procedure çağrılmaktadır. Bu procedure, kullanıcının program ve sıcaklık tercihine göre yıkama, durulama ve sıkma işlemlerine ait gecikme sürelerini belirlemekte; bu değerler sayaçların ulaşması gereken maksimum değerler olarak kullanılmak üzere ilgili sinyallere atanmaktadır.

Reset durumunda gerçekleştirilen işlemler Şekil 3.8'de verilmiştir.

```
if rst = '1' then

    program_sec_cikis <= "UUUU";
    cikis <= '0';
    sayici5 <=0;
    sayici3 <=0;
    sayici15 <=0;
    sayici_durula <=0;
    sayici_yika <=0;
    sayici_skma <=0;
```

Şekil 3. 8. Reset durumunda gerçekleştirilen atamalar

rst sinyalinin aktif olması durumunda, sistem başlangıç durumuna alınmaktadır. Program seçimi belirsizlik durumunu ifade eden değerlere atanmakta, çıkış sinyali pasif hale getirilmekte ve tüm sayaçlar sıfırlanmaktadır. Böylece sistem yeniden çalışmaya hazır hale getirilmektedir.

Saat darbesi ile tetiklenen normal çalışma koşulları Şekil 3.9'da gösterilmiştir.

```
elsif rising_edge(clk) then --clk yükselen kenarında

  if(kapak_kontrol='1') then --eğer kapak kapalıysa

        sonuc_eks_durulama <= durulama_sec;
        sonuc_on_yika <= on_yikama_sec;
        deterjan_var_mi <= deterjan_kontrol;

        simdiki_durum <= basla;
```

Şekil 3. 9. Kapak kapalıyken gerçekleştirilen başlangıç atamaları

Bu aşamada kapak_kontrol sinyali kontrol edilmekte; kapağın kapalı olması durumunda FSM çalışmaya başlamaktadır. Kullanıcının ön yıkama ve ekstra durulama tercihleri, giriş sinyallerinden okunarak FSM içerisinde kullanılmak üzere ara sinyallere atanmaktadır. Ayrıca deterjan kontrol sinyali, kullanıcıya deterjan var-yok durumu

hakkında geri bildirimde bulunabilmek amacıyla ayrı bir sinyale atanmaktadır. Tüm ön koşullar sağlandığında FSM, başlangıç durumu olan basla durumuna geçirilir.

FSM içerisindeki durum geçişleri, kontrol devrelerinde yaygın olarak kullanılan case yapısı ile gerçekleştirilmiştir. Case yapısı, FSM durumlarını temsil eden simdiki_durum sinyaline bağlı olarak çalışmakta ve her bir durum için gerçekleştirilecek işlemler ayrı ayrı tanımlanmaktadır.

Basla durumuna ait işlemler Şekil 3.10'da gösterilmiştir.

```
case simdiki_durum is

when basla =>
mevcut_sure_durumu <= 0;
simdiki_durum <= su_al1;
cikis <= '0';
```

Şekil 3. 10. basla durumuna ait işlem adımları

Bu durumda sayaçlar sıfırlanmakta ve FSM bir sonraki işlem adımı olan su_al1 durumuna geçirilmektedir. Program henüz tamamlanmadığı için çıkış sinyali pasif tutulmaktadır.

FSM akışındaki ilk kritik karar noktalarından biri su_al1 durumudur. Bu duruma ait işlem adımları Şekil 3.11'de gösterilmiştir.

```
when su_al1 =>
        if (sonuc_on_yika ='1') then
            if (sayici5 < max_sayici_deger5) then
            sayici5 <= sayici5 + 1;
            mevcut_sure_durumu <= sayici5;
            simdiki_durum <= su_al1;
            else
            cikis <= '0';
            mevcut_sure_durumu <= 5;
           sayici5 <= 0;
           simdiki_durum <= on_yika;
            end if;
        else --sonuc onyika 0 ise
        cikis <= '0';
        mevcut_sure_durumu <= 0;
        simdiki_durum <= on_yika;
        end if;
```

Şekil 3. 11. su_al1 durumunda yürütülecek işlemleri içeren kod bloğu

su_al1 durumu, ön yıkama işlemi öncesinde gerçekleştirilen ilk su alma aşamasını temsil etmektedir. Bu aşamada FSM, kullanıcının ön yıkama tercihini kontrol etmektedir. Eğer ön yıkama seçeneği aktif ise, su alma işlemi sayaç tabanlı bir gecikme yapısı ile gerçekleştirilmekte ve sayaç değeri belirlenen maksimum değere ulaşana kadar FSM bu durumda kalmaktadır. Sayaç değeri maksimum değere ulaştığında, sayaç sıfırlanmakta ve FSM bir sonraki durum olan on_yika aşamasına geçmektedir.

Ön yıkama seçeneğinin aktif olmadığı durumda ise, sayaç tabanlı bir işlem gerçekleştirilmemekte; FSM, doğrudan on_yika durumuna geçirilerek program akışı sade bir biçimde devam ettirilmektedir. Bu yapı sayesinde FSM, kullanıcı tercihlerine bağlı olarak farklı işlem yollarını güvenli ve kontrollü biçimde yönetebilmektedir.

FSM içerisinde yer alan diğer durumlar (on_yika, su_ver1, su_al2, yika, det_al, su_ver2, su_al3, durula, su_ver3, su_al4, eks_durula, su_ver4 ve skma) benzer sayaç tabanlı gecikme ve koşul kontrol mekanizmaları ile çalışmaktadır. Bu durumlara ait kod blokları sırasıyla Şekil 3.12 – Şekil 3.24 arasında sunulmuştur.

```
 when on_yika =>
if (sonuc_on_yika ='1') then
    if (sayici15 < max_sayici_deger15) then
            sayici15 <= sayici15 + 1;
            mevcut_sure_durumu <= sayici15;
            simdiki_durum <= on_yika;
            else
            sayici15 <= 0;
            cikis <= '0';
            mevcut_sure_durumu <= 15;
            simdiki_durum <= su_ver1;
            end if;
else
cikis <= '0';
simdiki_durum <= su_ver1;
end if;
```

Şekil 3. 12. on_yika durumunda yürütülecek işlemleri içeren kod bloğu

```
when su_ver1 =>
        if(sonuc_on_yika='1') then
            if (sayici5 < max_sayici_deger5) then
            sayici5 <= sayici5 + 1;
            mevcut_sure_durumu <= sayici5;
            simdiki_durum <= su_ver1;
            else
            sayici5 <= 0;
            cikis <= '0';
            mevcut_sure_durumu <= 5;
            simdiki_durum <= su_al2;
            end if;
        else
        cikis <= '0';
        mevcut_sure_durumu <= 0;
        simdiki_durum <= su_al2;
        end if;
```

Şekil 3. 13. su_ver1 durumunda yürütülecek işlemleri içeren kod bloğu

```
when su_al2 =>
        if (sayici15 < max_sayici_deger15) then
            sayici15 <= sayici15 + 1;
            mevcut_sure_durumu <= sayici15;
            simdiki_durum <= su_al2;
            else
            cikis <= '0';
            sayici15 <= 0;
            mevcut_sure_durumu <= 15;
            simdiki_durum <= yika;
            end if;
```

Şekil 3. 14. su_al2 durumunda yürütülecek işlemleri içeren kod bloğu

```
when yika =>
      if (sayici_yika < max_sayici_deger_yika) then
            sayici_yika <= sayici_yika + 1;
            mevcut_sure_durumu <= sayici_yika;
            simdiki_durum <= yika;
            else
            sayici_yika <= 0;
            cikis <= '0';
            mevcut_sure_durumu <= max_sayici_deger_yika;
            simdiki_durum <= det_al;
            end if;
```

Şekil 3. 15. yika durumunda yürütülecek işlemleri içeren kod bloğu

```
when det_al =>
        if (sayici3 < max_sayici_deger3) then
                sayici3 <= sayici3 + 1;
                mevcut_sure_durumu <= sayici3;
                simdiki_durum <= det_al;
                else
                sayici3 <= 0;
                cikis  <= '0';
                mevcut_sure_durumu <= 3;
                simdiki_durum <= su_ver2;
                end if;
```

Şekil 3. 16. det_al durumunda yürütülecek işlemleri içeren kod bloğu

```
when su_ver2 =>
            if (sayici15 < max_sayici_deger15) then
                sayici15 <= sayici15 + 1;
                mevcut_sure_durumu <= sayici15;
                simdiki_durum <= su_ver2;
                else
               sayici15 <= 0;
               cikis <= '0';
               mevcut_sure_durumu <= 15;
               simdiki_durum <= su_al3;
               end if;
```

Şekil 3. 17. su_ver2 durumunda yürütülecek işlemleri içeren kod bloğu

```
when su_al3 =>
            if (sayici15 < max_sayici_deger15) then
                sayici15 <= sayici15 + 1;
                mevcut_sure_durumu <= sayici15;
                simdiki_durum <= su_al3;
                else
                sayici15 <= 0;
                cikis <= '0';
                mevcut_sure_durumu <=  15;
                simdiki_durum <= durula;
                end if;
```

Şekil 3. 18. su_al3 durumunda yürütülecek işlemleri içeren kod bloğu

```
when durula =>
          if (sayici_durula < max_sayici_deger_durula) then
                  sayici_durula <= sayici_durula + 1;
                  mevcut_sure_durumu <= sayici_durula;
                  simdiki_durum <= durula;
                  else
                   sayici_durula <= 0;
                  cikis <= '0';
                  mevcut_sure_durumu <= max_sayici_deger_durula;
                  simdiki_durum <= su_ver3;
                  end if;
```

Şekil 3. 19. durula durumunda yürütülecek işlemleri içeren kod bloğu

```
when su_ver3 =>
                if (sayici15 < max_sayici_deger15) then
                    sayici15 <= sayici15 + 1;
                    mevcut_sure_durumu <= sayici15;
                    simdiki_durum <= su_ver3;
                    else
                    sayici15 <= 0;
                    cikis <= '0';
                    mevcut_sure_durumu <= 15;
                    simdiki_durum <= su_al4;
                    end if;
```

Şekil 3. 20. su_ver3 durumunda yürütülecek işlemleri içeren kod bloğu

```
when su_al4 =>
                if (sonuc_eks_durulama ='1') then
                   if (sayici5 < max_sayici_deger5) then
                    sayici5 <= sayici5 + 1;
                    mevcut_sure_durumu <= sayici5;
                    simdiki_durum <= su_al4;
                    else
                    sayici5 <= 0;
                    cikis <= '0';
                    mevcut_sure_durumu <= 5;
                    simdiki_durum <= eks_durula;
                    end if;
               else
                    cikis <= '0';
                    simdiki_durum <= eks_durula;
               end if;
```

Şekil 3. 21. su_al4 durumunda yürütülecek işlemleri içeren kod bloğu

```
when eks_durula =>
          if (sonuc_eks_durulama = '1' ) then
              if (sayici15 < max_sayici_deger15) then
                      sayici15 <= sayici15 + 1;
                      mevcut_sure_durumu <= sayici15;
                      simdiki_durum <= eks_durula;
                      else
                      sayici15 <= 0;
                      cikis <= '0';
                      mevcut_sure_durumu <= 15;
                      simdiki_durum <= su_ver4;
                      end if;
          else
          cikis <= '0';
          mevcut_sure_durumu <= 0;
          simdiki_durum <= su_ver4;
          end if;
```

Şekil 3. 22. eks_durula durumunda yürütülecek işlemleri içeren kod bloğu**.**

```
when su_ver4 =>
                  if(sonuc_eks_durulama='1') then
                      if (sayici5 < max_sayici_deger5) then
                      sayici5 <= sayici5 + 1;
                      mevcut_sure_durumu <= sayici5;
                      simdiki_durum <= su_ver4;
                      else
                      sayici5 <= 0;
                      cikis <= '0';
                      mevcut_sure_durumu <= 5;
                      simdiki_durum <= skma;
                      end if;
                  else
                  cikis <= '0';
                  mevcut_sure_durumu <= 0;
                  simdiki_durum <= skma;
                  end if;
```

Şekil 3. 23. su_ver4 durumunda yürütülecek işlemleri içeren kod bloğu

```
when skma =>
                if (sayici_skma < max_sayici_deger_skma) then
                        sayici_skma <= sayici_skma + 1;
                          mevcut_sure_durumu <= sayici_skma;
                          simdiki_durum <= skma;
                        else
                        sayici_skma <= 0;
                        mevcut_sure_durumu <= max_sayici_deger_skma;
                        cikis <= '1';
                        simdiki_durum <= basla;
                        end if;
```

Şekil 3. 24. skma durumunda yürütülecek işlemleri içeren kod bloğu

FSM'de yer alan others durumu, tanımlanmamış veya beklenmeyen bir durum oluşması halinde sistemin güvenli şekilde başlangıç durumuna dönmesini sağlamaktadır (Şekil 3.25).

```
when others =>
            cikis <= '0';
            mevcut_sure_durumu <= 0;
            simdiki_durum <= basla;
            end case;
```

Şekil 3. 25. others durumunda yürütülecek işlemleri içeren kod bloğu

Kapak açıkken gerçekleştirilen işlemler ise Şekil 3.26'da gösterilmiştir. Bu durumda sistem çalışması durdurulmakta, çıkış sinyali pasif hale getirilmekte ve program bilgileri belirsizlik durumuna alınarak sistem güvenli bekleme moduna geçirilmektedir.

```
                elsif(kapak_kontrol='0') then
                cikis <= '0';
                program_sec_cikis <= "UUUU";
                son_sicaklik <= "UUU";
               end if;--kapak_kontrol  için if
        end if; --if(rising_clk) için
    end process;
end davranis;
```

Şekil 3. 26. kapak açıksa yürütülecek işlemleri içeren kod bloğu

3.3. Testbench Mimarisi

Geliştirilen VHDL tabanlı çamaşır makinesi kontrol biriminin ana (main) tasarım kodlamasının tamamlanmasının ardından, sistemin doğru ve beklenen şekilde çalıştığını doğrulamak amacıyla bir testbench mimarisi oluşturulmuştur. Testbench yapısı, donanım tasarımının simülasyon ortamında gerçek çalışma koşullarına benzer senaryolar altında test edilmesini sağlayan ve tasarımın davranışının gözlemlenmesine imkan tanıyan bir doğrulama altyapısıdır.

Bu çalışma kapsamında oluşturulan testbench kodu aracılığıyla; kullanıcıdan alınan program seçimi, sıcaklık seçimi, ön yıkama ve ekstra durulama gibi giriş sinyallerinin sisteme uygulanması ve bu girişlere karşılık FSM yapısının doğru durum geçişlerini gerçekleştirip gerçekleştirmediği incelenmiştir. Ayrıca, zamanlama gereksinimlerini karşılamak üzere kullanılan sayaçların beklenen sürelerde çalışıp çalışmadığı ve program akışının doğru şekilde tamamlanıp tamamlanmadığı simülasyon ortamında doğrulanmıştır.

Testbench mimarisi, genel hatlarıyla kütüphane ve entity tanımlamaları, component yapısı, sinyal ve sabit tanımları, ana tasarımın testbench içerisinde örneklenmesi, stimulus süreçleri ile saat (clock) üretim mekanizmasından oluşmaktadır.

3.3.1. Kütüphane ve entity tanımlamaları

Testbench mimarisinin oluşturulmasında ilk adım olarak, VHDL dilinde kullanılacak kütüphane ve paket tanımlamaları gerçekleştirilmiştir. Bu tanımlar, testbench içerisinde kullanılacak veri türlerinin ve aritmetik işlemlerin sorunsuz bir şekilde yürütülmesini sağlamaktadır. Bu çalışmada, IEEE standartlarına ait kütüphaneler tercih edilmiş; mantıksal sinyallerin tanımlanması ve sayısal işlemlerin gerçekleştirilmesi amacıyla STD_LOGIC_1164 ve NUMERIC_STD paketleri testbench koduna dahil edilmiştir. Böylece, ana tasarımda kullanılan veri türleri ile testbench yapısı arasında tam uyumluluk sağlanmıştır.

Testbench'e ait entity tanımı, ana tasarımdan farklı olarak herhangi bir giriş veya çıkış portu içermemektedir. Bunun nedeni, testbench'in dış dünyadan veri almak yerine, test edilecek tasarımı (DUT – Device Under Test) kendi içerisinde tanımlanan sinyaller aracılığıyla sürmesidir. Bu yaklaşım, simülasyon senaryolarının kontrollü ve tekrarlanabilir biçimde uygulanmasına olanak tanımaktadır.

Testbench için oluşturulan kütüphane ve entity tanımlamalarına ait kod yapısı Şekil 3.27'de gösterilmiştir.

```
library IEEE;
use IEEE.Std_logic_1164.all;
use IEEE.Numeric_Std.all;

entity tanim_tb is
end;
```

Şekil 3. 27. Testbench için kütüphane, paket ve entity tanımlamalarını içeren kod bloğu

3.3.2. Architecture içerisinde yer alan component tanımlamaları

Testbench mimarisi içerisinde, test edilecek ana tasarımın simülasyon ortamına dahil edilebilmesi amacıyla bir component tanımı gerçekleştirilmiştir. Bu component yapısı, VHDL tabanlı çamaşır makinesi kontrol biriminin entity tanımı ile birebir uyumlu olacak şekilde oluşturulmuştur. Component tanımı sayesinde, ana tasarım

testbench mimarisi içerisinde bağımsız bir donanım bloğu gibi örneklenebilmekte ve giriş–çıkış sinyalleri testbench tarafından kontrol edilebilmektedir.

Tanımlanan component içerisinde; saat (clock) ve reset sinyallerinin yanı sıra kapak ve deterjan kontrol girişleri, program seçimi, ön yıkama ve ekstra durulama seçenekleri ile sıcaklık seçimine ait giriş sinyalleri yer almaktadır. Ayrıca, yıkama, durulama ve sıkma aşamalarına ait sayaç değerleri ile mevcut süre durumunu gösteren çıkışlar da simülasyon ortamında izlenebilmesi amacıyla component portları üzerinden erişilebilir durumdadır.

Testbench mimarisi içerisinde kullanılan component tanımlamasına ait kod yapısı Şekil 3.28'de sunulmuştur.

```
architecture bench of tanim_tb is

  component tanim
      Port (clk,rst : in std_logic;
      kapak_kontrol : in std_logic;
      deterjan_kontrol: in std_logic;
      program_sec_giris : in std_logic_vector(0 to 3);
      on_yikama_sec : in std_logic;
      durulama_sec : in std_logic;
      sicaklik_sec : in std_logic_vector(0 to 2);
      g_skma : inout integer :=0;
      g_durula : inout integer :=0 ;
      g_yika : inout integer :=0;
      mevcut_sure_durumu : out integer :=0;
      cikis : out std_logic);
  end component;
```

Şekil 3. 28. Testbench'e ait component yapısı

3.3.3. Architecture içerisinde yer alan sinyal ve sabit tanımlamaları

Testbench mimarisi içerisinde, ana tasarımın giriş ve çıkış portları ile uyumlu olacak şekilde çeşitli sinyal tanımlamaları yapılmıştır. Bu sinyaller, testbench tarafından üretilen uyarıların ana tasarıma iletilmesini ve tasarımdan elde edilen çıktıların simülasyon ortamında gözlemlenmesini sağlamaktadır.

Tanımlanan sinyaller; saat ve reset sinyallerinin yanı sıra kapak ve deterjan kontrolü, program seçimi, ön yıkama ve ekstra durulama tercihleri ile sıcaklık seçimine ait giriş sinyallerini kapsamaktadır. Bununla birlikte, yıkama, durulama ve sıkma aşamalarına ait sayaç değerleri ile mevcut süre durumunu temsil eden sinyaller de testbench içerisinde izlenebilmesi amacıyla tanımlanmıştır.

Ayrıca, saat sinyalinin periyodunun belirlenebilmesi amacıyla bir zaman sabiti tanımlanmış; simülasyon sürecinin kontrollü bir şekilde sonlandırılabilmesi için clock üretimini durdurmaya yönelik bir kontrol sinyali kullanılmıştır. Testbench architecture bloğu içerisinde yer alan sinyal ve sabit tanımlarına ait kod yapısı Şekil 3.29'da gösterilmiştir.

```
  signal clk,rst: std_logic;
  signal kapak_kontrol: std_logic;
  signal deterjan_kontrol: std_logic;
  signal program_sec_giris: std_logic_vector(0 to 3);
  signal on_yikama_sec: std_logic;
  signal durulama_sec: std_logic;
  signal sicaklik_sec: std_logic_vector(0 to 2);
  signal g_skma: integer :=0;
  signal g_durula: integer :=0;
  signal g_yika: integer :=0;
  signal cikis: std_logic;
  signal mevcut_sure_durumu : integer := 0;

  constant clock_period: time := 2 ns;
  signal stop_the_clock: boolean;

begin
```

Şekil 3. 29. Testbench architecture sinyal ve sabit tanımlarını içeren kod bloğu

3.3.4. Ana tasarımın testbench içerisinde örneklenmesi (Port map)

Testbench mimarisi içerisinde tanımlanan component yapısının çalıştırılabilmesi amacıyla, ana tasarım testbench architecture bloğu içerisinde örneklenmiştir. Bu işlem, VHDL dilinde port map yapısı kullanılarak gerçekleştirilmiştir. Port map ifadesi

sayesinde, ana tasarımın giriş ve çıkış portları ile testbench içerisinde tanımlanan sinyaller birebir eşleştirilmiştir.

Bu eşleştirme ile testbench tarafından üretilen saat, reset ve kontrol sinyalleri ana tasarıma aktarılmış; ana tasarımın ürettiği çıkışlar ve sayaç değerleri simülasyon ortamında gözlemlenebilir hâle getirilmiştir. Ana tasarımın testbench içerisinde örneklenmesine ait port map yapısı Şekil 3.30'da sunulmuştur.

```
uut: tanim port map ( clk                => clk,
                      rst                => rst,
                      kapak_kontrol      => kapak_kontrol,
                      deterjan_kontrol   => deterjan_kontrol,
                      program_sec_giris  => program_sec_giris,
                      on_yikama_sec      => on_yikama_sec,
                      durulama_sec       => durulama_sec,
                      sicaklik_sec       => sicaklik_sec,
                      g_skma             => g_skma,
                      g_durula           => g_durula,
                      g_yika             => g_yika,
                      mevcut_sure_durumu => mevcut_sure_durumu,
                      cikis              => cikis );
```

Şekil 3. 30. Testbench port map yapısını içeren kod bloğu

3.3.5. Uyarı (Stimulus) süreci ve örnek durumlar üzerinden gözlem

Testbench mimarisi içerisinde, tasarımın farklı çalışma senaryolarının simülasyon ortamına aktarılabilmesi amacıyla bir uyarı (stimulus) süreci tanımlanmıştır. Bu süreç kapsamında, farklı kullanım durumlarını temsil eden giriş sinyalleri testbench içerisinde sıralı olarak uygulanmıştır.

Simülasyonun başlangıç aşamasında, tasarımın başlangıç durumuna alınabilmesi amacıyla reset sinyalinin uygulandığı stimulus kod bloğu Şekil 3.31'de gösterilmiştir. Reset işleminin ardından, tez kapsamında örnek olarak pamuklu programın 20 °C

sıcaklıkta, ön yıkama ve ekstra durulama seçenekleri aktif olacak şekilde tanımlandığı senaryoya ait stimulus kod bloğu Şekil 3.32'de sunulmuştur.

Testbench içerisinde, örneklenen durumlar dışındaki diğer tüm program, sıcaklık ve ek özellik kombinasyonları için de benzer yapılar oluşturulmuştur. Ancak tekrar eden kod bloklarının önüne geçmek amacıyla, tez kapsamında yalnızca örnek olarak seçilen bu iki senaryonun kod yapısına yer verilmiştir.

```
stimulus: process
begin

  -- Put initialisation code here


  ------------------------------
  -- reset durumu
  rst <= '1';
  wait for 40 ns;
  rst <= '0';
  wait for 40 ns;
```

Şekil 3. 31. Testbench stimulus - reset durumu kod bloğu

```
------------------------------
-- pamuklu 20 derece ön yıkama ve ekstra durulama var durumu
kapak_kontrol     <= '1';
deterjan_kontrol <= '1';
durulama_sec      <= '1';
on_yikama_sec     <= '1';
program_sec_giris <= "0000";
sicaklik_sec       <= "000";
wait for 375 ns;
```

Şekil 3. 32. Testbench stimulus - pamuklu 20 °C programı için ön yıkama ve ekstra durulama seçeneklerinin aktif olduğu durum kod bloğu

3.3.6. Saat (Clock) üretimi ve simülasyonun sonlandırılması

Testbench mimarisi içerisinde, ana tasarımın çalışabilmesi için gerekli olan saat (clock) sinyali ayrı bir süreç (process) içerisinde üretilmiştir. Clock sinyalinin periyodu, daha önce tanımlanan zaman sabiti kullanılarak belirlenmiş ve simülasyon süresince sürekli olarak üretilmesi sağlanmıştır.

Testbench içerisinde clock sinyalinin üretildiği ve simülasyonun kontrol edildiği kod bloğu Şekil 3.33'te gösterilmiştir. Stimulus süreci içerisinde tanımlanan tüm senaryolar tamamlandıktan sonra, stop_the_clock sinyali aktif hâle getirilerek clock üretimi durdurulmuş ve simülasyon süreci kontrollü bir şekilde sonlandırılmıştır. Bu yaklaşım, simülasyonun gereksiz yere devam etmesini engelleyerek testbench akışının düzenli ve yönetilebilir olmasını sağlamaktadır.

```
    -- Put test bench stimulus code here

    stop_the_clock <= true;
    wait;
  end process;

  clocking: process
  begin
    while not stop_the_clock loop
      clk <= '0', '1' after clock_period / 2;
      wait for clock_period;
    end loop;
    wait;
  end process;

end;
```

Şekil 3. 33. Testbench içerisinde clock sinyalinin üretildiği ve simülasyonun kontrol edildiği kod bloğu

4. ARAŞTIRMA (veya DENEY) SONUÇLARI

Main kod ve testbench kodlamasının tamamlanmasının ardından, geliştirilen VHDL tabanlı çamaşır makinesi kontrol biriminin sistem davranışını incelemek amacıyla simülasyon çalışmaları gerçekleştirilmiştir. Bu simülasyonlar, tasarlanan kontrol yapısının donanımsal karşılığının ve zaman içerisindeki sinyal değişimlerinin gözlemlenebilmesini amaçlamaktadır.

Bu kapsamda, öncelikle tasarıma ait RTL (Register Transfer Level) devre yapısı oluşturularak sistemin genel donanım mimarisi incelenmiştir. Ardından, oluşturulan testbench mimarisi kullanılarak farklı çalışma senaryolarına ait dalga formları (waveform) elde edilmiş ve sistemin tanımlanan giriş koşullarına verdiği tepkiler analiz edilmiştir.

Bu bölümde sunulan değerlendirmelerde, kontrol biriminin tüm program, sıcaklık ve ek özellik kombinasyonları için simülasyon çalışmaları gerçekleştirilmiştir. Ancak, tez içerisinde tekrar eden yapıların önüne geçmek ve anlatım bütünlüğünü korumak amacıyla, elde edilen sonuçlar arasından sistemin temel işleyiş mantığını ve kontrol yapısını temsil eden seçili örnek durumlara yer verilmiştir. Diğer senaryolara ait simülasyon sonuçları da incelenmiş olup, bu sonuçların benzer çalışma prensipleri sergilediği gözlemlenmiştir.

Bu bölümde, simülasyon sürecine ait sonuçlar iki temel başlık altında sunulmaktadır. İlk olarak tasarlanan kontrol birimine ait RTL devre yapısı verilmiş, ardından testbench aracılığıyla elde edilen simülasyon dalga formları örnek durumlar üzerinden incelenmiştir.

4.1. RTL Devre Yapısı

Bu bölümde, geliştirilen VHDL tabanlı çamaşır makinesi kontrol biriminin RTL analizinden elde edilen devre yapısı sunulmaktadır. RTL analizi sonucunda oluşturulan şemalar, tasarlanan sistemin donanımsal karşılığını ve alt bloklar arasındaki yapısal ilişkileri ortaya koymaktadır.

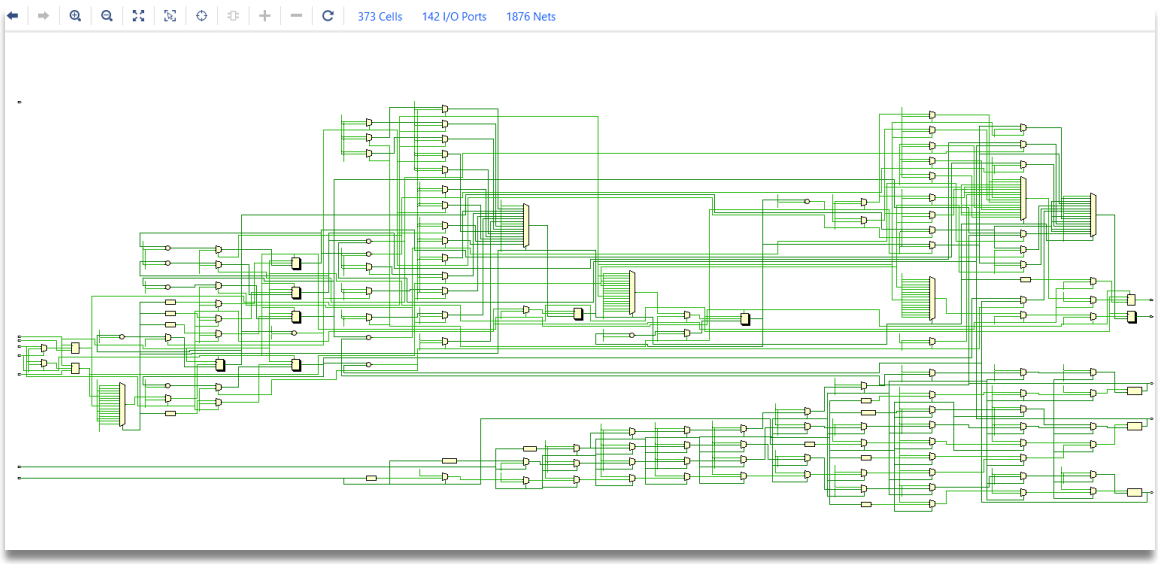
Kontrol birimi; FSM yapısı, sayaç blokları ve zamanlama kontrol mantıklarını içeren kapsamlı bir yapıya sahiptir. Bu nedenle RTL devre yapısı tez içerisinde hem genel görünüm hem de alt bloklar bazında parça parça sunulmuştur. Bu yaklaşım sayesinde, sistemin karmaşık yapısının daha anlaşılır bir biçimde incelenmesi amaçlanmıştır.

Bu kapsamda, öncelikle kontrol birimine ait genel RTL şeması verilmiş; ardından FSM yapısı ile sayaç ve kontrol bloklarına ait RTL görünümleri ayrı alt başlıklar altında ele alınmıştır.

4.1.1. Genel RTL şeması

Şekil 4.1'de, geliştirilen çamaşır makinesi kontrol birimine ait RTL analizinden elde edilen genel devre yapısı sunulmuştur. Bu şema, sistemin tüm alt bloklarını ve bu bloklar arasındaki bağlantıları genel hatlarıyla göstermektedir.

RTL şeması incelendiğinde; FSM yapısı, süre kontrolü için kullanılan sayaç blokları, kullanıcı girişlerine bağlı kontrol mantıkları ve çıkış sinyallerinin oluşturulmasına yönelik veri yollarının tek bir yapı içerisinde bütünleşik olarak yer aldığı görülmektedir. Bu yapı, kontrol biriminin modüler ve senkron bir donanım mimarisi ile tasarlandığını ortaya koymaktadır.



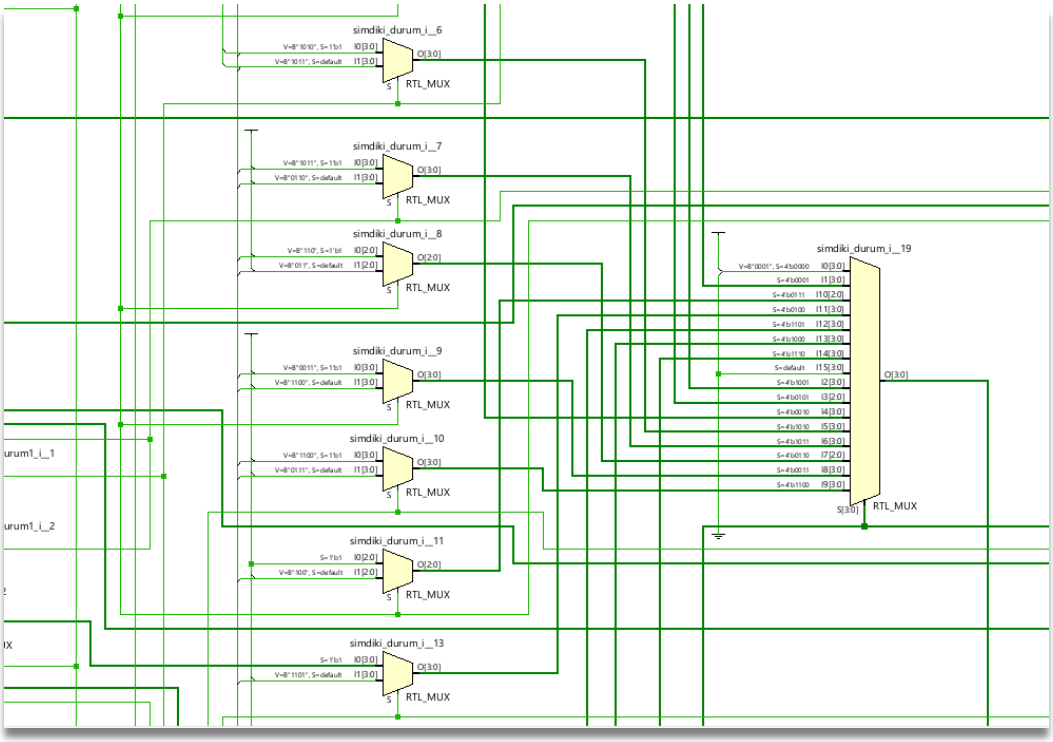
Şekil 4. 1. RTL analizi ile elde edilen devre yapısı

4.1.2. FSM yapısına ait RTL görünümü

Şekil 4.2'de, çamaşır makinesi kontrol biriminin durum geçişlerini yöneten sonlu durum makinesi (FSM) yapısına ait RTL analizinden elde edilen devre görünümünün bir kısmı sunulmuştur. Bu görünüm, sistemde tanımlanan durumların donanımsal karşılığını ve durumlar arasındaki geçiş mantığını oluşturan yapıların genel olarak incelenebilmesine olanak sağlamaktadır.

FSM yapısına ait RTL görünümünde, mevcut durumu temsil eden sinyallerin çoklayıcılar (MUX) aracılığıyla değerlendirildiği ve bir sonraki duruma geçişin bu yapı üzerinden belirlendiği görülmektedir. Durum geçişleri, farklı kontrol sinyalleri ve koşullara bağlı olarak seçilmekte olup bu seçim işlemleri donanım seviyesinde çoklayıcı yapılarla gerçekleştirilmektedir.

Bu RTL görünümü, yazılım tarafında tanımlanan FSM durum geçişlerinin donanım ortamında birebir karşılık bulduğunu ve sistemin işlem adımlarının donanımsal olarak doğru şekilde modellenmiş olduğunu göstermektedir.



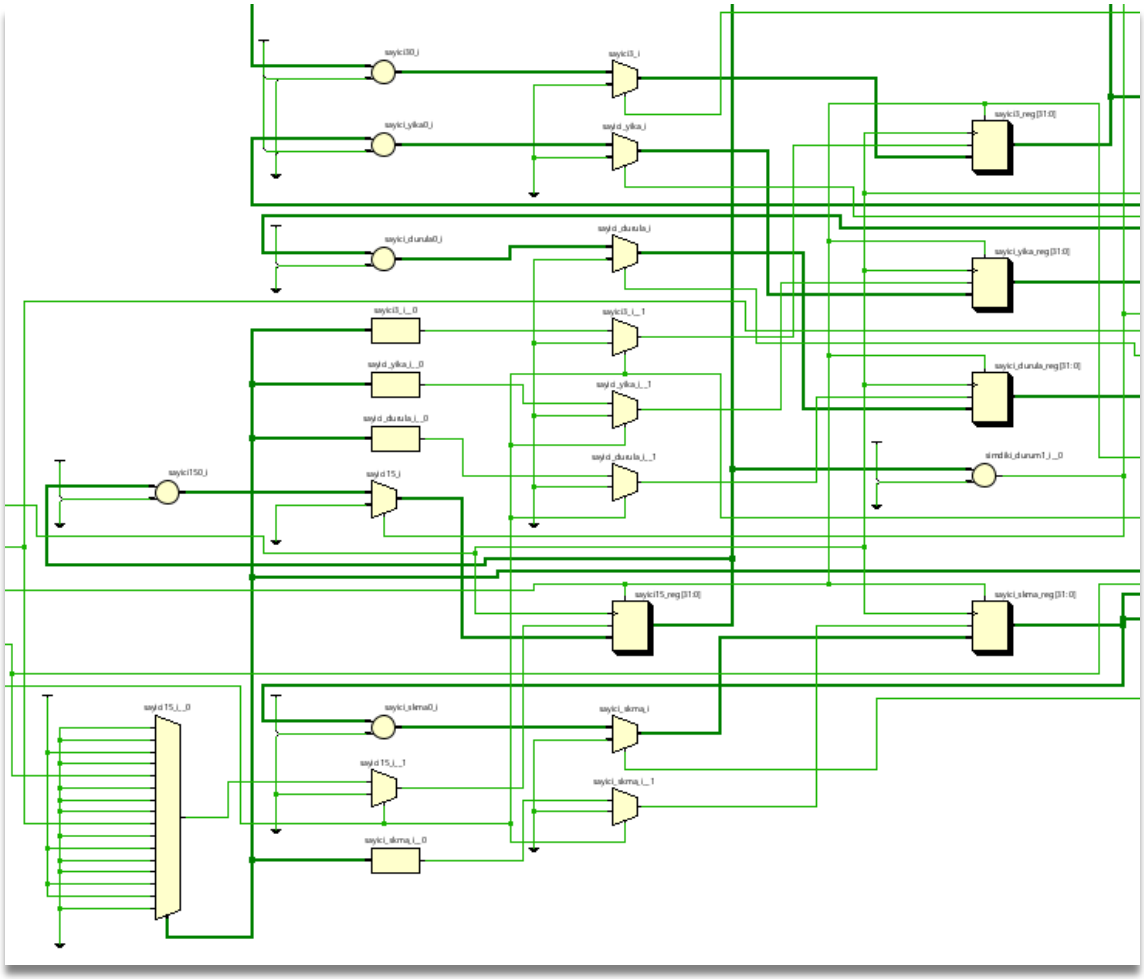
Şekil 4. 2. FSM yapısına ait RTL devre örneği

4.1.3. Sayaç ve kontrol bloklarına ait RTL görünümü

Şekil 4.3'te, çamaşır makinesi kontrol birimi içerisinde yer alan sayaç ve zamanlama kontrol bloklarına ait RTL analiz görünümünün bir kısmı sunulmuştur. Bu yapı, FSM işlem adımlarında süre kontrolünün sağlanması amacıyla kullanılan sayaç mekanizmalarının donanımsal karşılığını göstermektedir.

RTL görünümünde, farklı işlem adımlarına ait süre bilgilerini tutan sayaç register'ları ile bu sayaçların artış, sıfırlama ve kontrol işlemlerini yöneten mantık blokları yer almaktadır. Sayaç girişlerinin, mevcut durum bilgileri ve kontrol sinyallerine bağlı olarak çoklayıcılar üzerinden seçildiği görülmektedir. Bu yapı sayesinde, her bir program ve işlem adımı için gerekli zamanlama bilgisi dinamik olarak belirlenebilmektedir.

Ayrıca, sayaç değerlerinin karşılaştırılması ve belirlenen eşik değerlere ulaşıldığında FSM durum geçişlerinin tetiklenmesini sağlayan kontrol yapıları da RTL seviyesinde açıkça gözlemlenmektedir. Bu durum, zaman tabanlı FSM geçişlerinin donanım ortamında doğru şekilde gerçekleştirildiğini ortaya koymaktadır.



Şekil 4. 3. Sayaç ve kontrol bloklarına ait RTL devre örneği

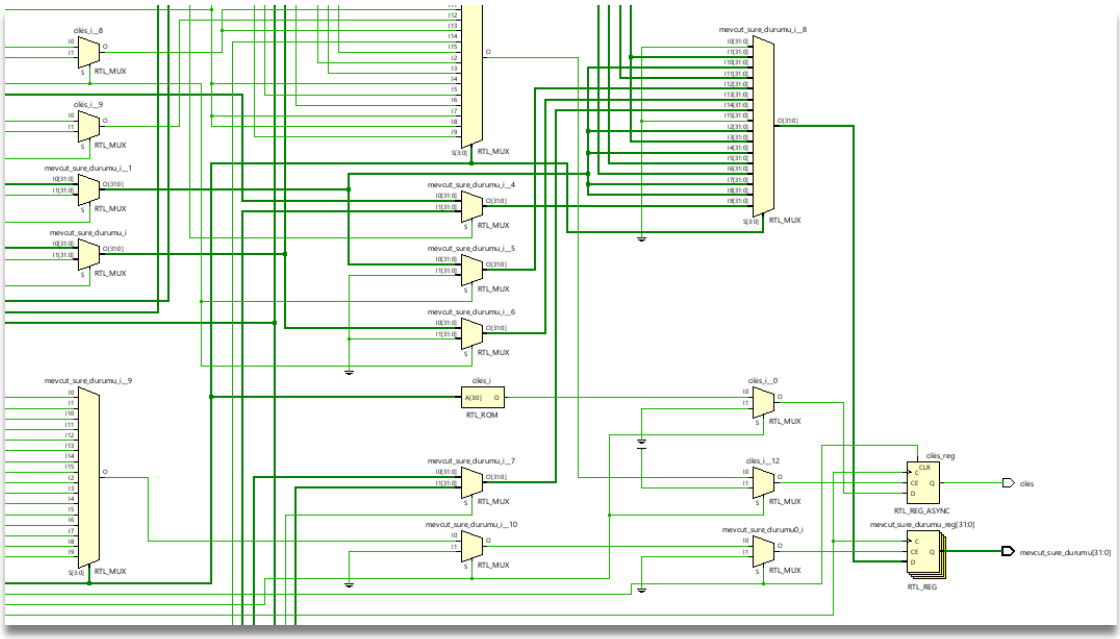
4.1.4. mevcut_sure_durumu'nun RTL seviyesinde oluşturulması

Şekil 4.4'te, çamaşır makinesi kontrol birimi içerisinde kullanıcıya bilgi vermek amacıyla üretilen mevcut_sure_durumu sinyaline ait RTL analiz görünümünün bir kısmı sunulmuştur. Bu yapı, sistemin anlık olarak hangi işlem adımında bulunduğunu ve ilgili adıma ait süre bilgisini temsil eden donanımsal yapıyı göstermektedir.

RTL görünümünde, mevcut_sure_durumu sinyalinin; FSM tarafından belirlenen mevcut durum bilgileri, ilgili sayaç değerleri ve kontrol sinyalleri doğrultusunda çoklayıcılar aracılığıyla oluşturulduğu görülmektedir. Farklı durumlara ait süre bilgileri,

mevcut durum sinyaline bağlı olarak seçilmekte ve uygun veri yolu üzerinden birleştirilmektedir.

Oluşturulan süre bilgisi, senkron bir register yapısı üzerinden tutulmakta ve sistem çıkışı olarak aktarılmaktadır. Bu yapı sayesinde, süre bilgisinin kararlı bir şekilde saklanması ve saat sinyali ile senkron olarak güncellenmesi sağlanmıştır.



Şekil 4. 4. Mevcut_sure_durumu'na ait RTL devre örneği

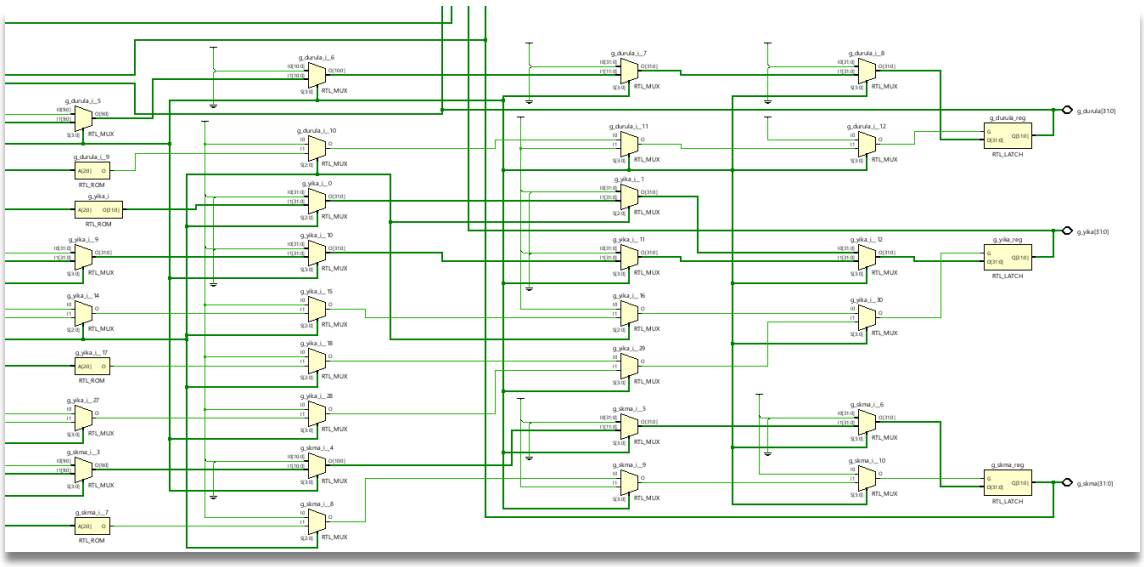
4.1.5. Kullanıcı seçimlerine göre yıkama, durulama ve sıkma sürelerinin RTL seviyesinde belirlenmesi

Şekil 4.5'te, kullanıcıdan alınan program seçimi, sıcaklık bilgisi ve ek özelliklere bağlı olarak yıkama (*g_yika*), durulama (*g_durula*) ve sıkma (*g_skma*) işlemlerine ait süre değerlerinin belirlendiği RTL analiz görünümünün bir kısmı sunulmuştur. Bu yapı, sistemde tanımlanan farklı program kombinasyonlarına karşılık gelen süre bilgilerinin donanım seviyesinde nasıl oluşturulduğunu göstermektedir.

RTL görünümünde, kullanıcı girişleri ve FSM durum bilgilerine bağlı olarak farklı süre değerlerinin çoklayıcılar aracılığıyla seçildiği görülmektedir. Program türü, sıcaklık

seviyesi ve ek özelliklerin aktif olup olmaması gibi parametreler, ilgili süre bilgisinin belirlenmesinde doğrudan etkili olmaktadır.

Seçilen süre değerleri, her bir işlem adımı için ayrı yapılar üzerinden tutulmakta ve *g_yika*, *g_durula* ve *g_skma* çıkışları olarak sistem dışına aktarılmaktadır. Bu sayede, farklı kullanıcı tercihlerine bağlı olarak değişen süre konfigürasyonlarının donanım seviyesinde güvenilir ve kararlı bir şekilde oluşturulduğu gözlemlenmektedir.



Şekil 4. 5. Yıkama, durulama, sıkma durumlarına ait RTL devre örneği

4.2. Testbench Simülasyon Sonuçları ve Dalga Formlarının İncelenmesi

Bu bölümde, geliştirilen VHDL tabanlı çamaşır makinesi kontrol biriminin testbench ortamında gerçekleştirilen simülasyon sonuçları sunulmuştur. Testbench mimarisi içerisinde tanımlanan uyarı (stimulus) sinyalleri kullanılarak, sistemin farklı çalışma durumlarına ait dalga formları elde edilmiş ve bu çıktılar üzerinden kontrol biriminin zamana bağlı davranışı incelenmiştir.

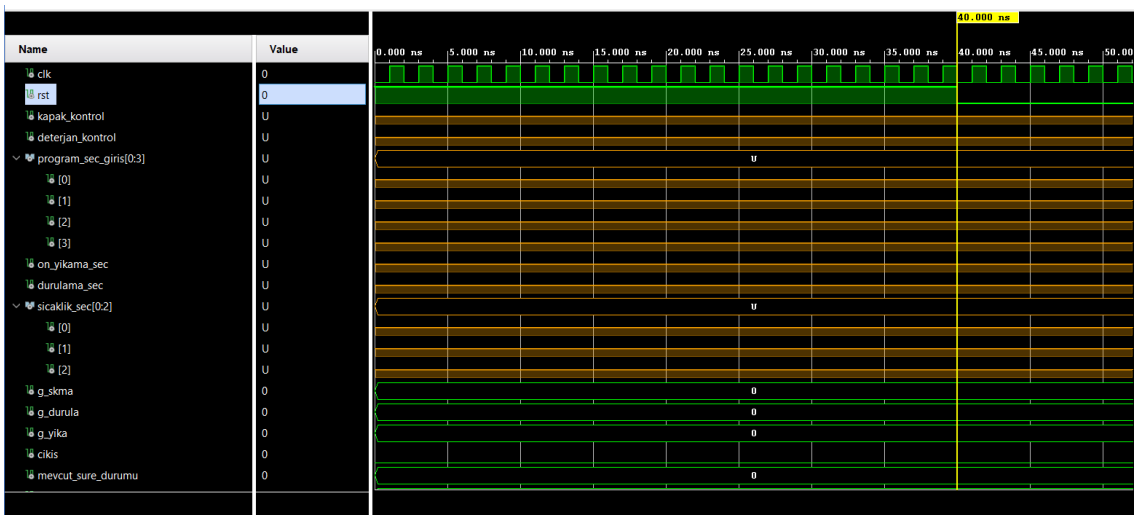
Simülasyon sürecinde, tasarımın başlangıç koşullarındaki davranışını gözlemlemek amacıyla reset durumu ile kullanıcı tarafından seçilen program ve ek özelliklerin sistem üzerindeki etkisini temsil eden çeşitli çalışma senaryoları test edilmiştir. Bu kapsamda,

kontrol biriminin durum geçişleri, süre bilgileri ve çıkış sinyallerinin zaman içerisindeki değişimi dalga formları üzerinden takip edilmiştir.

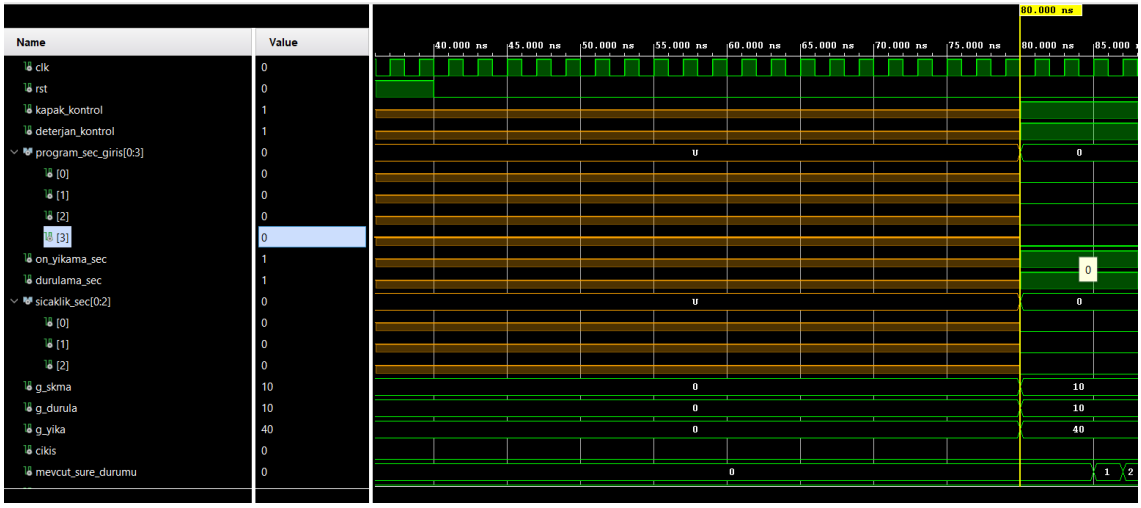
Testbench yapısı, sistemde tanımlı tüm program, sıcaklık ve ek özellik kombinasyonlarını kapsayacak şekilde oluşturulmuş ve bu senaryoların tamamı simülasyon ortamında test edilmiştir. Ancak, tez içerisinde tekrar eden yapıların önüne geçmek ve anlatım bütünlüğünü korumak amacıyla, elde edilen simülasyon sonuçları arasından kontrol algoritmasının ve zamanlama mekanizmasının doğruluğunu temsil eden seçili örnek durumlar sunulmuştur. Reset durumu ile pamuklu programının 20 °C sıcaklıkta, ön yıkama ve ekstra durulama seçenekleri aktif olacak şekilde çalıştırıldığı senaryo, bu kapsamda örnek olarak ele alınmıştır.

İlerleyen alt başlıklarda, seçilen bu senaryolara ait sistem tepkileri dalga formu çıktıları üzerinden ayrıntılı olarak incelenmiştir.

4.2.1. Reset durumu simülasyon sonuçları



Şekil 4. 6. Reset durumu simülasyon çıktısı – 1



Şekil 4. 7. Reset durumu simülasyon çıktısı - 2

Şekil 4.6 ve Şekil 4.7'de, çamaşır makinesi kontrol birimine ait reset durumunun testbench simülasyonu sonucunda elde edilen dalga formları sunulmuştur. Bu simülasyon çıktıları, sistemin başlangıç koşullarındaki davranışını ve reset sinyalinin kontrol birimi üzerindeki etkisini incelemek amacıyla değerlendirilmiştir.

Dalga formlarından görüldüğü üzere, reset sinyalinin aktif olduğu (rst = '1') süre boyunca, yıkama (g_yika), durulama (g_durula) ve sıkma (g_skma) işlemlerine ait süre değerleri sıfıra çekilmekte; mevcut_sure_durumu sinyali herhangi bir sayma işlemi gerçekleştirmeden sıfır değerinde tutulmaktadır. Buna karşılık, program seçimi, ön yıkama, ekstra durulama ve sıcaklık seçimine ait giriş sinyallerinin belirsizliği temsil eden 'U' değerinde kaldığı gözlemlenmektedir. Bu durum, reset aktifken sistemin kullanıcı girişlerini dikkate almadığını ve güvenli bir başlangıç durumuna zorlandığını göstermektedir.

Simülasyon dalga formlarında, reset sinyalinin testbench içerisinde tanımlandığı şekilde yaklaşık 40 ns boyunca aktif, ardından 40ns boyunca pasif duruma geçtiği açıkça görülmektedir. Reset sinyalinin pasif hâle gelmesiyle birlikte sistem, saat sinyali ile senkron olarak normal çalışma moduna geçmeye hazır duruma gelmektedir.

Elde edilen bu simülasyon sonuçları, ana tasarım kodu içerisinde tanımlanan reset davranışının testbench ortamında doğru şekilde gerçekleştirildiğini ve kontrol biriminin başlangıç koşullarına uygun biçimde çalıştığını doğrulamaktadır.

4.2.2. Pamuklu programı (20°C, ön yıkama ve ekstra durulama aktif) simülasyon sonuçları

Bu alt bölümde, pamuklu programı için 20 °C sıcaklık değeri seçilmiş, ön yıkama ve ekstra durulama seçeneklerinin aktif olduğu bir çalışma senaryosuna ait testbench simülasyon sonuçları incelenmiştir. Sunulan dalga formları, FSM tabanlı kontrol biriminin kullanıcı girişlerine bağlı olarak nasıl bir işlem akışı izlediğini ve sayaç temelli zamanlama mekanizmasının doğruluğunu göstermek amacıyla değerlendirilmiştir.

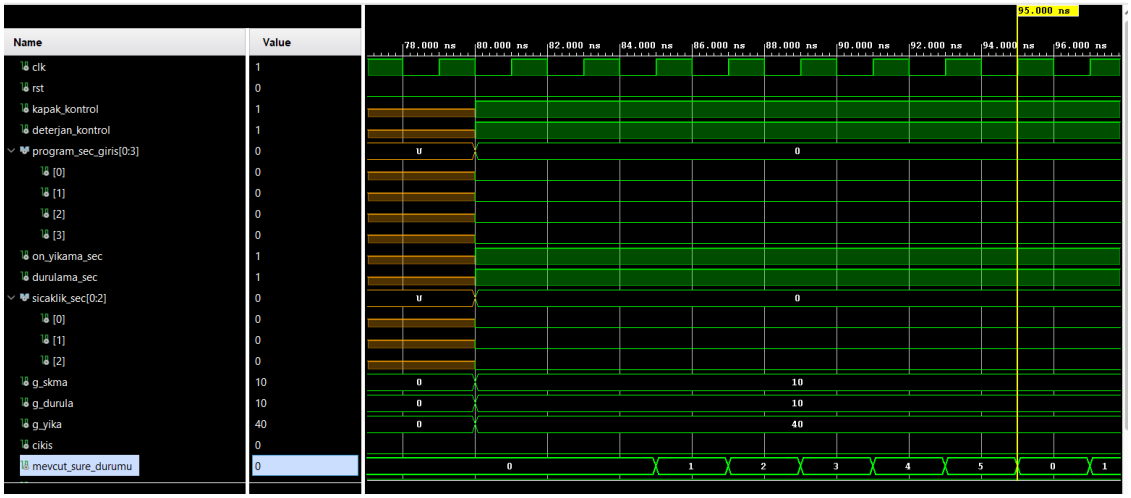
Şekil 4.8'de, simülasyonun başlangıç aşaması olan ön yıkama öncesi su alma durumuna ait dalga formları sunulmuştur. Dalga formlarından görüldüğü üzere, kapak_kontrol sinyali '1' seviyesinde olup kapak kapalı durumdadır. Benzer şekilde deterjan_kontrol sinyali '1' değerini alarak deterjanın mevcut olduğunu göstermektedir. program_sec_giris sinyalinin "0000" değerinde olması pamuklu programın seçildiğini, on_yikama_sec ve durulama_sec sinyallerinin '1' olması ise ön yıkama ve ekstra durulama seçeneklerinin aktif edildiğini göstermektedir. sicaklik_sec sinyalinin "000" değerine ayarlanmasıyla sıcaklık seçimi 20 °C olarak belirlenmiştir.

Bu giriş koşulları altında, simülasyon çıktılarında yıkama süresinin 40, durulama ve sıkma sürelerinin ise 10 olarak belirlendiği gözlemlenmektedir. Elde edilen süre değerleri, ana tasarım kodu içerisinde ilgili program ve ek özellikler için tanımlanan gecikme değerleri ile birebir uyumludur. Dalga formunun zaman ekseni incelendiğinde, simülasyonun ilk 80 ns'lik bölümü reset durumuna karşılık geldiğinden resetin pasif hâle gelmesinin ardından FSM'nin normal çalışma akışına geçtiği görülmektedir. Başlangıç aşamasında gerekli atamalar yapıldığından, mevcut_sure_durumu sinyalinin kısa bir süre sıfır değerinde kaldığı gözlemlenmektedir.

Bu aşamada, ön yıkama seçeneği aktif olduğu için su alma işlemi sayaç tabanlı olarak gerçekleştirilmiş; sayaç belirlenen süre (5 tetikleme) sonunda sıfırlanarak FSM bir sonraki duruma geçirilmiştir. Böylece, su alma süresinin tanımlanan gecikme değerine uygun şekilde kontrol edildiği doğrulanmıştır.

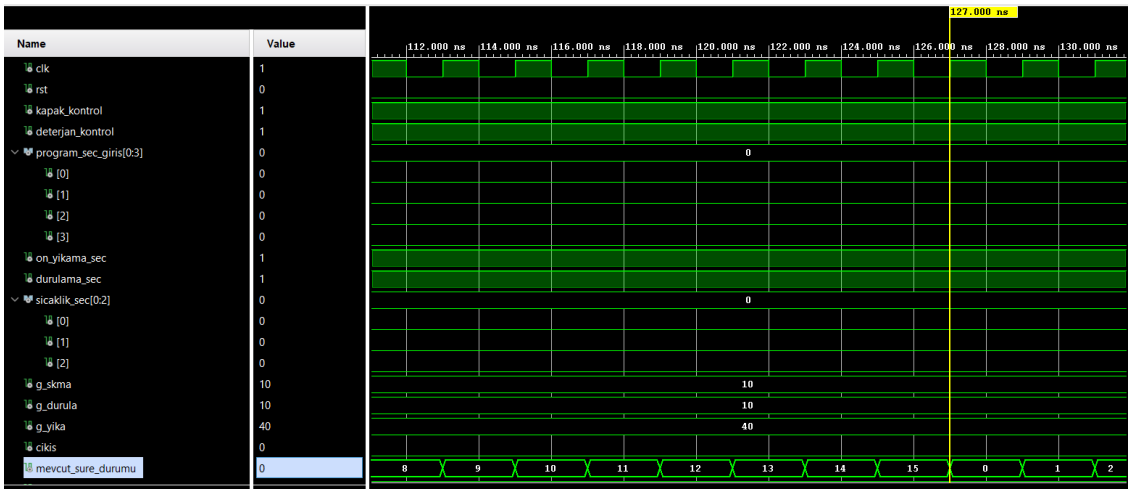
Ön yıkama seçeneğinin pasif olması durumunda ise FSM bu duruma yine girecek, ancak sayaç tabanlı herhangi bir süre sayımı yapılmadan yalnızca tek bir saat tetiklemesi süresince yeni duruma atama gerçekleştirilecekti. Bu yaklaşım, FSM

yapısının kullanıcı tercihlerine bağlı olarak koşullu ve verimli bir şekilde çalıştığını göstermektedir.



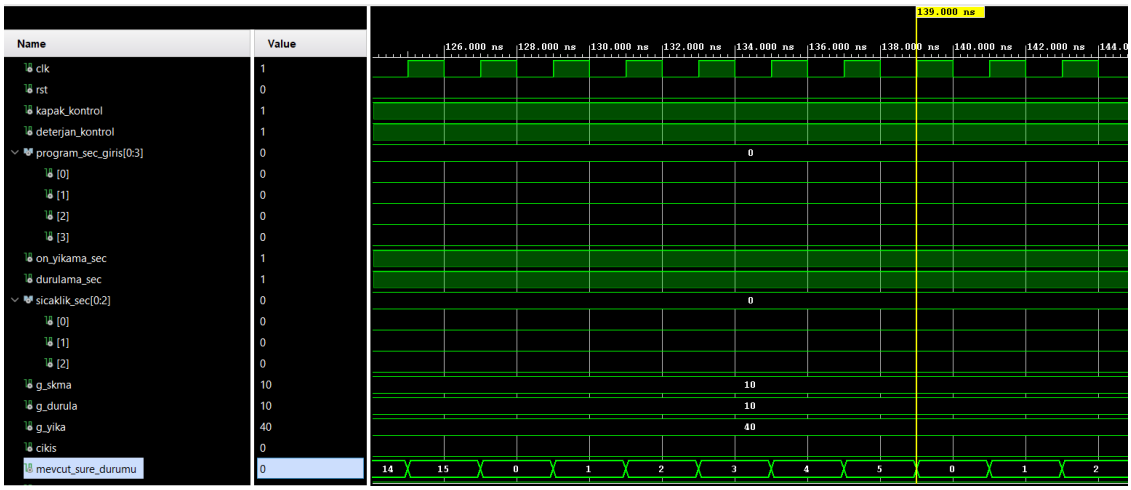
Şekil 4. 8. Ön yıkama öncesi su alma aşamasına ait dalga formu

Şekil 4.9'da, ön yıkama öncesi su alma işleminin tamamlanmasının ardından FSM'nin ön yıkama işlemini gerçekleştirdiği görülmektedir. Dalga formunda, ön yıkama işlemi için tanımlanan gecikme süresi (15 tetikleme) boyunca mevcut_sure_durumu sayacının arttığı ve belirlenen süre tamamlandığında sayaç değerinin sıfırlanarak FSM'nin bir sonraki durum olan su verme işlemine geçiş yaptığı açıkça gözlemlenmektedir. Bu davranış, ana tasarım kodunda tanımlanan ön yıkama süresi ve durum geçiş mekanizması ile uyumludur.



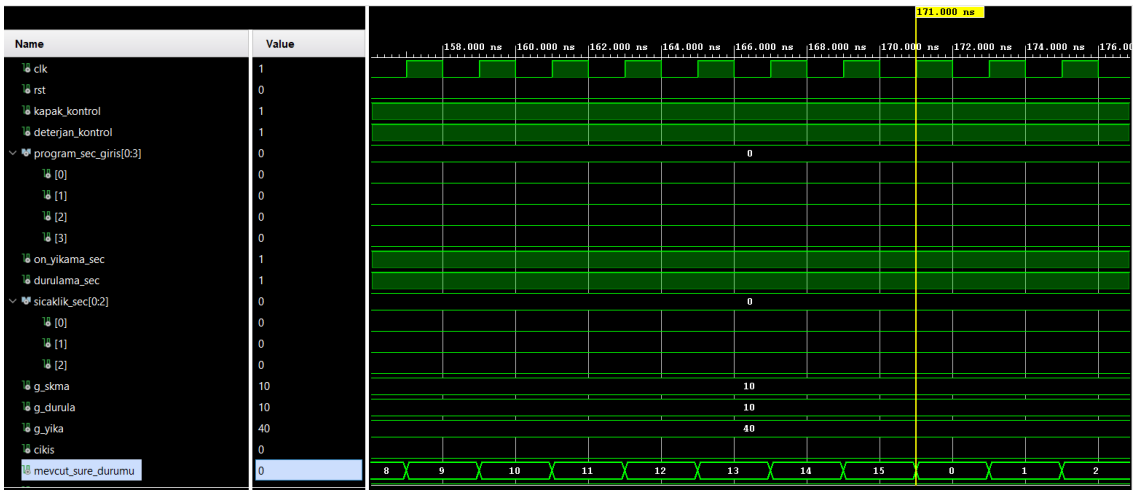
Şekil 4. 9. Ön yıkama aşamasına ait dalga formu

Şekil 4.10'da, ön yıkama işleminin tamamlanmasının ardından sisteme alınan suyun tahliye edilmesi amacıyla gerçekleştirilen su verme durumuna ait FSM davranışı sunulmuştur. Sayaç değerinin tanımlanan süre (5 tetikleme) boyunca arttığı ve sürenin tamamlanmasıyla birlikte FSM'nin bir sonraki durum olan ana yıkama işlemine geçtiği görülmektedir.



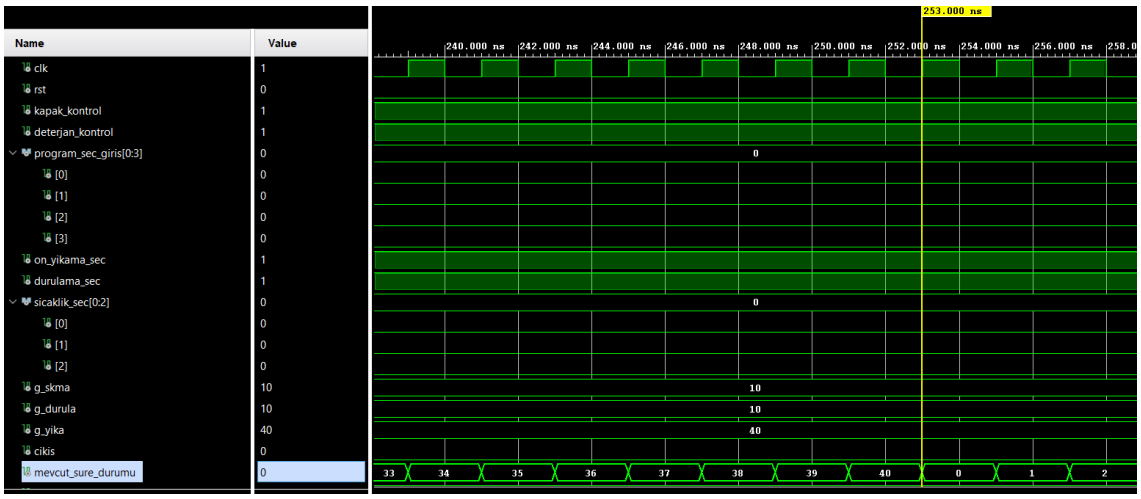
Şekil 4. 10. Ön yıkama sonrası su tahliye aşamasına ait dalga formu

Şekil 4.11'de, ön yıkama sonrası ana yıkama sürecinin başlatılabilmesi için gerekli olan temiz su alma aşamasına ait simülasyon çıktısı yer almaktadır. Bu aşama için tanımlanan sabit gecikme süresinin (15 tetikleme) sayaç tabanlı olarak doğru şekilde kontrol edildiği ve ardından yeni durum için sıfırlandığı gözlemlenmektedir.



Şekil 4. 11. Ana yıkama öncesi temiz su alma aşamasına ait dalga formu

Şekil 4.12'de, pamuklu programı ve 20 °C sıcaklık değeri için ana yıkama işleminin gerçekleştirildiği FSM durumu görülmektedir. Yıkama süresi boyunca mevcut_sure_durumu sayacının kesintisiz olarak arttığı ve belirlenen süreye (40 tetikleme) ulaşıldığında sayacın sıfırlandığı, FSM'nin bir sonraki durum olan deterjan alma işlem adımına geçtiği açıkça gözlemlenmektedir.

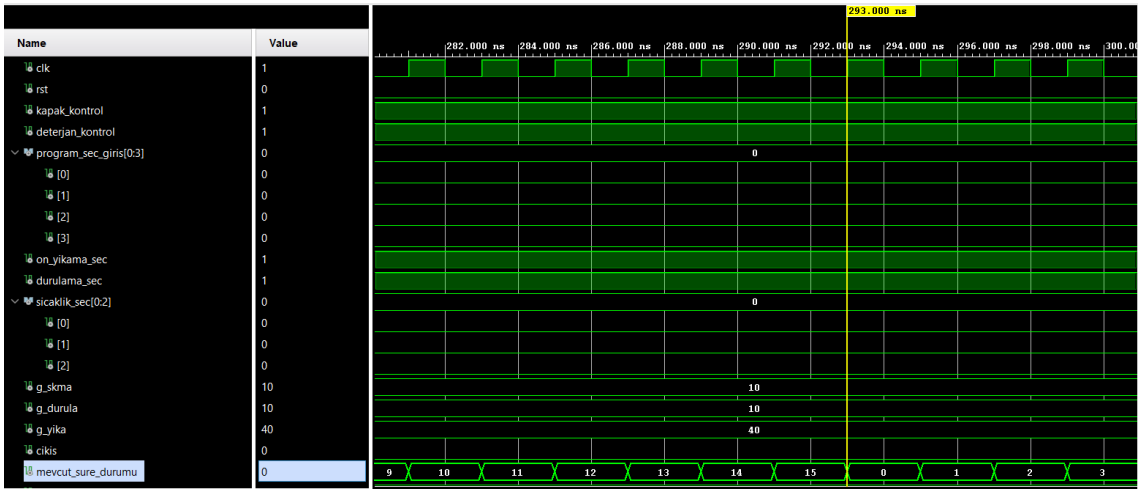


Şekil 4. 12. Ana yıkama aşamasına ait dalga formu

Şekil 4.13'te, FSM'nin deterjan alma işlemini gerçekleştirdiği görülmektedir. Bu aşama için tanımlanan gecikme süresinin (3 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan su verme işlem adımına geçiş yaptığı doğrulanmaktadır.

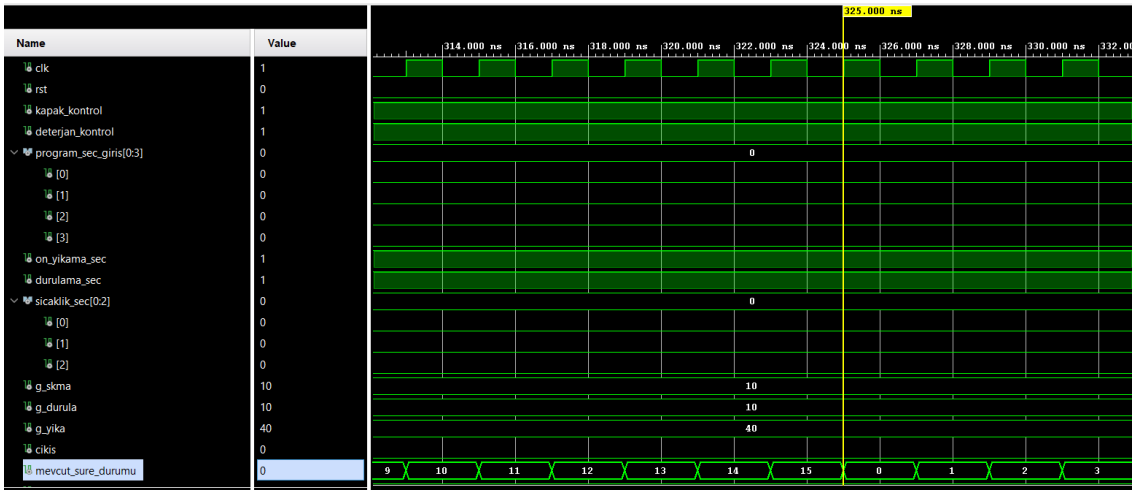
Şekil 4. 13. Deterjan alma aşamasına ait dalga formu

Şekil 4.14'te, yıkama ve deterjan alma işlemlerinin ardından gerçekleştirilen su verme aşamasına ait FSM davranışı sunulmuştur. Bu aşama için tanımlanan gecikme süresinin (15 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan durulama öncesi su alma işlem adımına geçiş yaptığı doğrulanmaktadır.



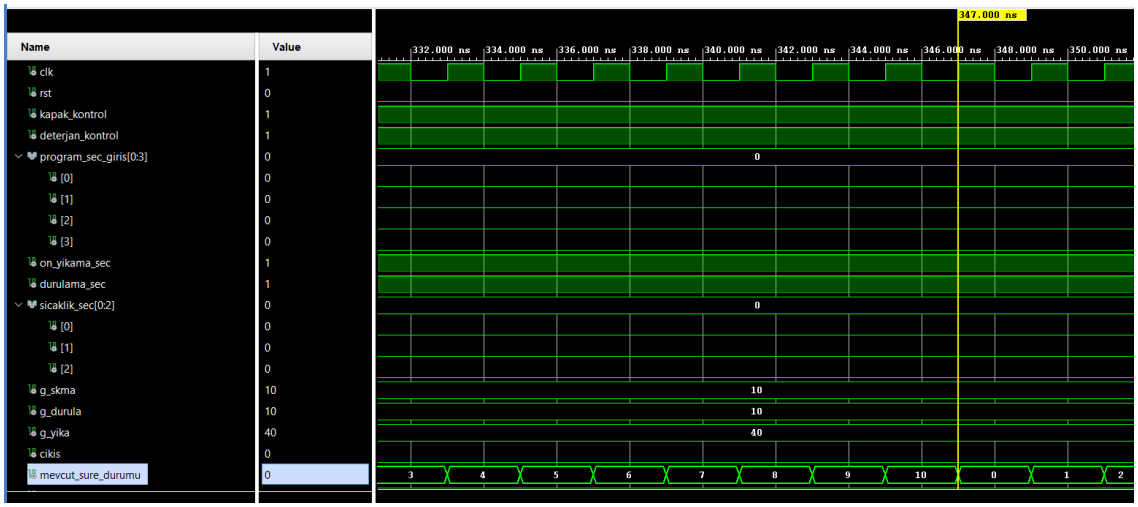
Şekil 4. 14. Yıkama sonrası su tahliye aşamasına ait dalga formu

Şekil 4.15'te, durulama işleminin başlatılabilmesi için gerekli olan su alma aşamasına ait simülasyon çıktısı yer almaktadır. Bu aşama için tanımlanan gecikme süresinin (15 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan durulama işlem adımına geçiş yaptığı doğrulanmaktadır.



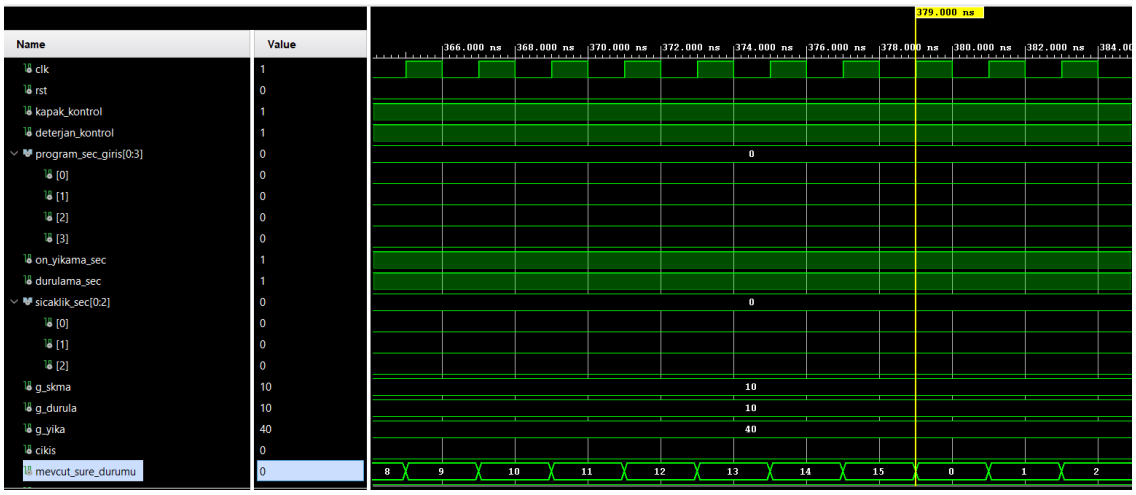
Şekil 4. 15. Durulama öncesi su alma aşamasına ait dalga formu

Şekil 4.16'da, durulama işleminin aktif olarak gerçekleştirildiği FSM durumu sunulmuştur. Bu aşama için tanımlanan gecikme süresinin (10 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan su verme işlem adımına geçiş yaptığı doğrulanmaktadır.



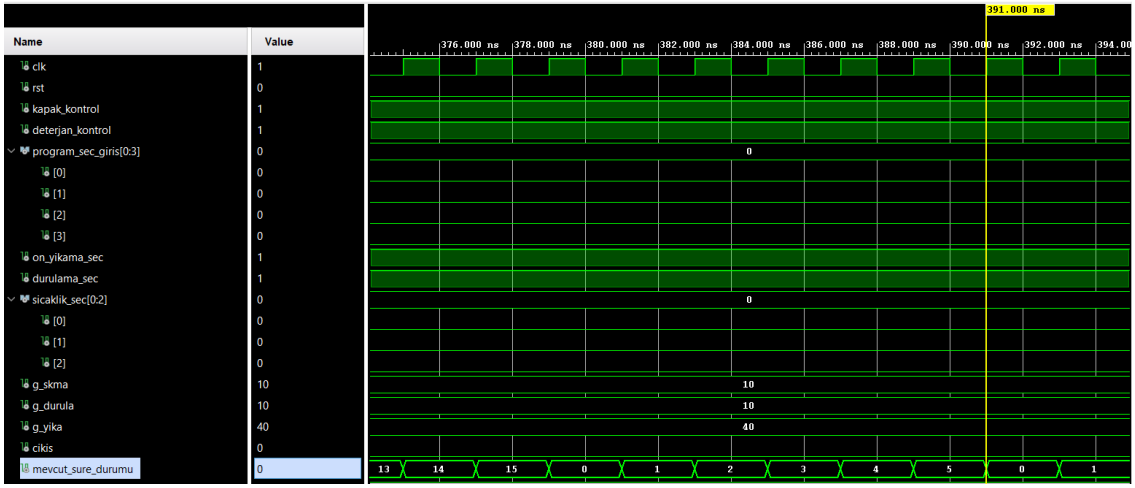
Şekil 4. 16. Durulama aşamasına ait dalga formu

Şekil 4.17'de, durulama işlemi sonrasında, sisteme alınan suyun tahliye edildiği aşamaya ait FSM davranışı gösterilmektedir. Bu aşama için tanımlanan gecikme süresinin (15 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan su verme işlem adımına geçiş yaptığı doğrulanmaktadır.



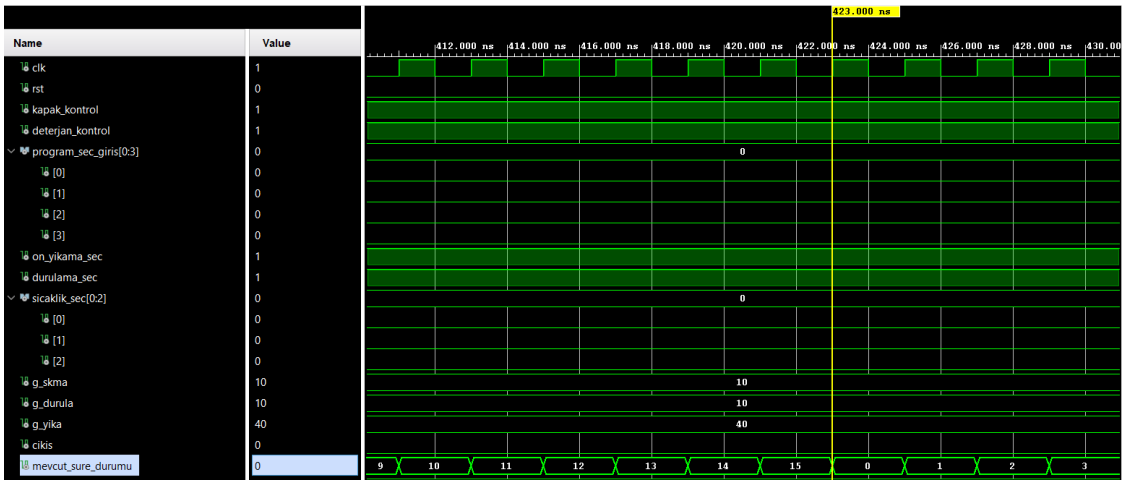
Şekil 4. 17. Durulama sonrası su tahliye aşamasına ait dalga formu

Şekil 4.18'de, ekstra durulama seçeneğinin aktif olması nedeniyle FSM'nin bu işleme ait su alma durumunda işlem gerçekleştirdiği gözlemlenmektedir. Bu aşama için tanımlanan gecikme süresinin (5 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan ekstra durulama işlem adımına geçiş yaptığı doğrulanmaktadır.



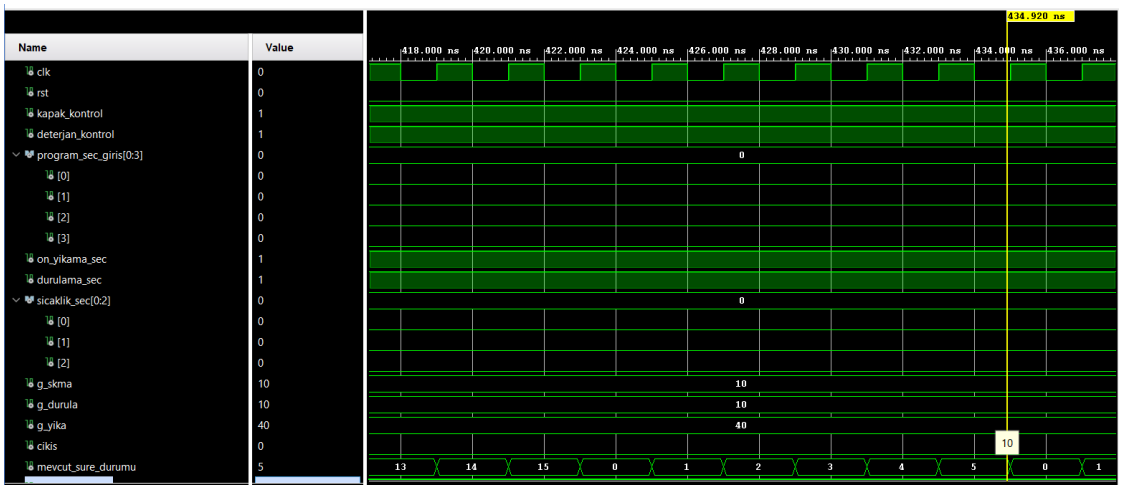
Şekil 4. 18. Ekstra durulama için su alma aşamasına ait dalga formu

Şekil 4.19'da, ekstra durulama işleminin gerçekleştirildiği FSM durumu sunulmuştur. Bu aşama için tanımlanan gecikme süresinin (15 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan su verme işlem adımına geçiş yaptığı doğrulanmaktadır.



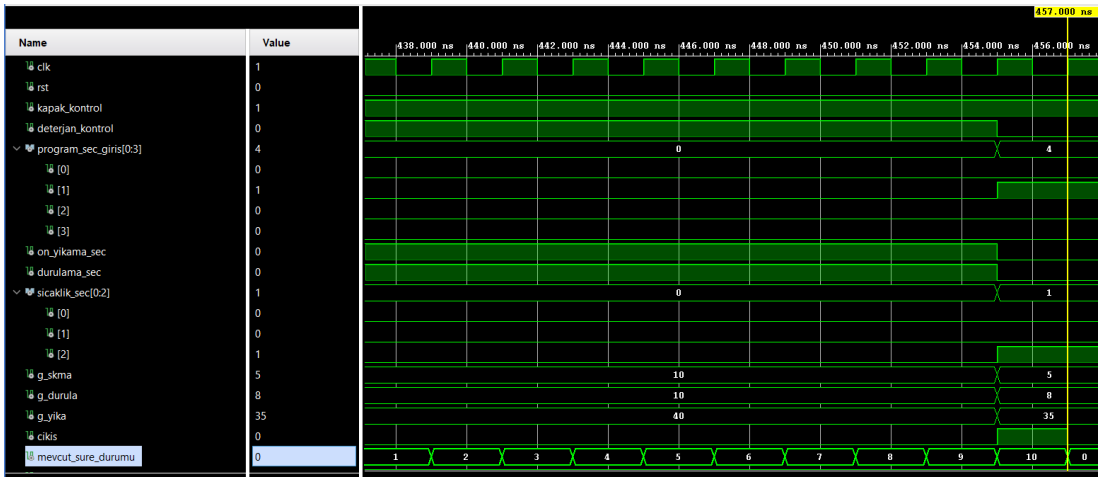
Şekil 4. 19. Ekstra durulama aşamasına ait dalga formu

Şekil 4.20'de, ekstra durulama işlemi sonrasında gerçekleştirilen su verme aşamasına ait simülasyon çıktısı yer almaktadır. Bu aşama için tanımlanan gecikme süresinin (5 tetikleme) sayaç üzerinden doğru şekilde sayıldığı ve süre sonunda FSM'nin bir sonraki durum olan su verme işlem adımına geçiş yaptığı doğrulanmaktadır.



Şekil 4. 20. Ekstra durulama sonrası su tahliye aşamasına ait dalga formu

Şekil 4.21'de, programın son aşaması olan sıkma işlemi ve programın tamamlanmasına ait FSM davranışı görülmektedir. Sıkma süresinin (10 tetikleme) tamamlanmasının ardından cikis sinyalinin '1' seviyesine geçtiği ve programın başarıyla tamamlandığı açıkça gözlemlenmektedir.



Şekil 4. 21. Sıkma aşaması ve program bitişine ait dalga formu

5. TARTIŞMA

Bu çalışmada geliştirilen çamaşır makinesi kontrol birimi, literatürde yaygın olarak kullanılan mikrodenetleyici tabanlı kontrol yaklaşımlarından farklı bir bakış açısıyla ele alınmıştır. Çoğu çalışmada kontrol algoritmaları gömülü yazılım seviyesinde tanımlanırken, bu tez kapsamında sistem davranışı FSM tabanlı ve VHDL kullanılarak doğrudan donanım seviyesinde modellenmiştir. Bu yaklaşım, kontrol algoritmasının yapısal ve zamansal özelliklerinin açık bir biçimde ifade edilmesine olanak tanımaktadır.

Literatürde çamaşır makinesi kontrolü genellikle mikrodenetleyici tabanlı çözümlerle gerçekleştirilmekte, durum geçişleri ve zamanlama yazılım tabanlı sayaçlar ve koşullu ifadeler aracılığıyla yönetilmektedir. Bu tür sistemlerde zamanlama doğruluğu işlemci yükü, yazılım akışı ve kesmeler gibi faktörlere bağlı olarak değişkenlik gösterebilmektedir. Ayrıca, kontrol algoritmasının donanımsal karşılığı çoğu zaman açık bir şekilde modellenmemekte ve algoritmanın iç yapısı soyut bir düzeyde kalmaktadır.

Bu çalışmada tercih edilen FSM tabanlı donanımsal yaklaşım sayesinde, her bir işlem adımının süresi ve durum geçiş koşulları donanım tarafından kesin olarak belirlenmiştir. Süre yönetimi, yazılım tabanlı zamanlayıcılara bağlı kalınmaksızın sayaç mekanizmaları ile gerçekleştirilmiş; böylece sistem davranışı deterministik ve izlenebilir bir yapı kazanmıştır. Bu durum, literatürde sıklıkla karşılaşılan "kontrol algoritmasının zamanlama yapısının açıkça gösterilememesi" problemini önemli ölçüde ortadan kaldırmaktadır.

Literatürde yer alan birçok çalışmada, kontrol algoritmalarının yalnızca işlevsel sonuçlarına odaklanıldığı, durum geçişlerinin ve zamanlama mantığının sınırlı düzeyde paylaşıldığı görülmektedir. Bu tez kapsamında ise ana kontrol birimi ile birlikte testbench yapısı da ayrıntılı biçimde modellenmiş ve sistemin farklı senaryolardaki davranışı dalga formları üzerinden analiz edilmiştir. Bu yönüyle çalışma, çamaşır makinesi kontrol algoritmasının yalnızca çalıştığını göstermekle kalmayıp, nasıl ve hangi zamanlama mantığıyla çalıştığını da ortaya koymaktadır.

Elde edilen doğrulamalar simülasyon ortamı ile sınırlıdır ve fiziksel bir FPGA uygulaması bu çalışma kapsamında gerçekleştirilmemiştir. Ancak bu durum, çalışmanın kapsamı dahilinde bilinçli bir tercih olarak değerlendirilmelidir. Simülasyon sonuçları, FSM tabanlı donanımsal kontrol yaklaşımının teorik olarak tutarlı ve uygulanabilir olduğunu göstermekte; fiziksel donanım uygulamasına geçilmesi durumunda karşılaşılabilecek zamanlama ve durum geçişlerinin önceden analiz edilmesine imkan sağlamaktadır.

Sonuç olarak, bu çalışma literatürde ağırlıklı olarak mikrodenetleyici tabanlı çözümlerle ele alınan bir kontrol probleminin, FPGA ve donanımsal tasarım bakış açısıyla da ele alınabileceğini göstermektedir. Geliştirilen FSM tabanlı yapı, özellikle donanımsal kontrol algoritmalarının öğretimi, zamanlama analizlerinin yapılması ve sistem davranışının şeffaf biçimde modellenmesi açısından önemli bir örnek sunmaktadır. Bu yönüyle çalışma, ileride gerçekleştirilebilecek fiziksel FPGA uygulamaları ve sistem genişletmeleri için sağlam ve analiz edilebilir bir altyapı oluşturmaktadır.

6. SONUÇ VE ÖNERİLER

Bu çalışmada, çamaşır makinesi kontrol algoritması FSM tabanlı bir yaklaşımla VHDL kullanılarak donanımsal düzeyde tasarlanmış ve testbench ortamında simülasyonlar aracılığıyla doğrulanmıştır. Geliştirilen kontrol birimi; program seçimi, sıcaklık ayarı, ön yıkama, durulama, ekstra durulama ve sıkma gibi temel işlem adımlarını durum tabanlı bir yapı ile yönetmekte, süre kontrolü ise sayaç mekanizmaları aracılığıyla gerçekleştirilmektedir. Elde edilen simülasyon sonuçları, tasarlanan FSM yapısının tanımlanan giriş senaryolarına karşılık olarak beklenen davranışları sergilediğini ve zamanlama açısından tutarlı bir çalışma ortaya koyduğunu göstermektedir.

Çalışma kapsamında tercih edilen FSM yapısında, bazı işlem adımlarının kullanıcı tarafından seçilmemesi durumunda dahi ilgili duruma geçilmekte; ancak bu durum içerisinde herhangi bir işlem gerçekleştirilmeden doğrudan bir sonraki duruma atama yapılmaktadır. Bu yaklaşım, FSM yapısının daha sade ve anlaşılır olmasını sağlamış, durum geçiş mantığının bütüncül bir yapı içerisinde ele alınmasına olanak tanımıştır.

Bununla birlikte, bu tasarım tercihinin bir sonucu olarak, işlem yapılmayan bu durumlara girilmesi ve hemen ardından yeni duruma geçilmesi sırasında toplamda iki saat darbesi kadar gecikme oluştuğu gözlemlenmiştir. Bu gecikme; ilgili işlem adımına giriş yapılması ve işlem gerçekleştirilmeden yeni duruma geçilmesi sırasında oluşan birer saat darbesinden kaynaklanmaktadır. Simülasyon ortamında bu gecikme sistemin genel işleyişini bozacak düzeyde olmamakla birlikte, zaman hassasiyetinin kritik olduğu uygulamalarda dikkate alınması gereken bir durumdur.

Bu doğrultuda, ileride yapılacak çalışmalarda FSM yapısının daha hassas bir zamanlama sağlayacak şekilde geliştirilmesi önerilmektedir. Özellikle, kullanıcıdan okunan giriş değerlerine bağlı olarak uygulanmayacak işlem adımlarına hiç girilmeden doğrudan ilgili bir sonraki duruma geçiş yapılması, gereksiz saat darbesi gecikmelerinin önüne geçilmesini sağlayacaktır. Bu yaklaşım, FSM yapısını daha karmaşık hâle getirse de zamanlama açısından daha verimli bir kontrol mekanizması sunacaktır.

Ayrıca, bu çalışmada gerçekleştirilen tüm doğrulamalar simülasyon ortamında ve belirli bir saat darbe periyodu (clock frekansı) varsayımı altında yapılmıştır. Tasarımın gerçek bir FPGA üzerinde fiziksel olarak uygulanması durumunda, kullanılan FPGA modeline ve seçilen saat frekansına bağlı olarak işlem süreleri ve zamanlama davranışları değişiklik gösterebilecektir. Bu nedenle, gerçek donanım uygulamalarında saat frekansına bağlı süre parametrelerinin yeniden ölçeklendirilmesi ve sistem gereksinimlerine uygun zamanlama ayarlamalarının yapılması önerilmektedir.

Sonuç olarak, bu çalışma FSM tabanlı ve donanımsal bir çamaşır makinesi kontrol biriminin başarılı bir şekilde modellenebileceğini göstermiştir. Geliştirilen yapı, FPGA tabanlı kontrol sistemleri için eğitsel ve araştırma amaçlı bir referans niteliği taşımakta olup, ileride yapılabilecek fiziksel donanım uygulamaları ve sistem genişletmeleri için sağlam bir temel oluşturmaktadır.

7. KAYNAKÇA

[1] Bosch Hausgeräte GmbH, *WAW28760TR Çamaşır Makinesi Kullanım Kılavuzu*, Bosch, Almanya.
" https://share.google/IxlbaoC6l2dWMIYX5 "

[2] Bosch Hausgeräte GmbH, *9001481865_A Çamaşır Makinesi Kullanım Kılavuzu*, Bosch, Almanya.
" https://share.google/fzDSiXppGBKOXu4RW "

[3] Arçelik A.Ş., *7103 D Çamaşır Makinesi Kullanım Kılavuzu*, Arçelik, Türkiye.
" https://share.google/05mhn3qvlgrITGIUV "

[4] Stilevs Elektrikli Ev Aletleri, *ST-WAC0310KG1501EU(A3) Çamaşır Makinesi Kullanım Kılavuzu*, Stilevs, Türkiye.
" https://share.google/05mfJtU7icROG2eQk "

[5] Beko A.Ş., *CMXD 9100 Çamaşır Makinesi Kullanım Kılavuzu*, Beko, Türkiye.
" https://share.google/hEIlytxKM6HL6Pi5D "

[6] LG Electronics Inc., *F4V3VYW3WE.ABWPLTK Çamaşır Makinesi Kullanım Kılavuzu*, LG Electronics, Güney Kore.
" https://gscs-b2c.lge.com/open/downloadFile?fileId=pqJSRXB81vGb2j8P1sCdw "

[7] Ferdjallah, M., *Introduction to Digital Systems: Modeling, Synthesis, and Simulation Using VHDL*, Springer, New York, ABD, 2011. Erişim: Perlego dijital kütüphanesi.

[8] Palekar, T., Jose, R. T., Nanche, G. ve Joshi, R., "VHDL Kullanarak Beş Katlı Asansör Kontrol Cihazının İncelenmesi ve Simülasyonu," *2019 3. Uluslararası Elektronik ve Bilişim Trendleri Konferansı (ICOEI)*, Tirunelveli, Hindistan, 2019, ss. 1270–1273, doi: 10.1109/ICOEI.2019.8862702.

[9] Sklyarov, V., “Hiyerarşik Sonlu Durum Makineleri ve Dijital Kontrol için Kullanımları,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, cilt 7, no. 2, ss. 222–228, Haziran 1999, doi: 10.1109/92.766749.

[10] Pedroni, V. A., *Finite State Machines in Hardware: Theory and Design*, MIT Press, Cambridge, ABD, 2013.

[11] Kazanmak, M. T., “VHDL Kodlama Teknikleri ile Dizi Dedektörü Kullanılarak Mealy ve Moore Durum Modellerinin Karşılaştırılması,” *International Journal of Advanced Research in Computer Engineering & Technology (IJARCET)*, cilt 7, ss. 651–656, 2018.

EKLER

Proje dosyasına:

https://drive.google.com/drive/folders/1r9b5Z5gbp5Kooax7ri4zwziWbomhI2O3?usp=sharing

linkinden ulaşabilirsiniz.